



TRƯỜNG ĐẠI HỌC QUY NHƠN

BÀI TIỂU LUẬN CUỐI KỲ HỌC PHẦN PHÂN TÍCH THỐNG KÊ VỚI R

BOOTSTRAP VÀ K-FOLD CROSS-VALIDATION TRONG BÀI TOÁN DỰ ĐOÁN BỆNH GAN

Trình độ đào tạo: Thạc sĩ
Học viên thực hiện: Nguyễn Hồ Bảo Thiên
Lớp: Khoa học dữ liệu K28B
Giảng viên hướng dẫn: TS. Nguyễn Đặng Thiên Thu

Gia Lai, 2026

Mục lục

1	Đặt vấn đề	3
2	Cơ sở lý thuyết	4
2.1	Phương pháp Bootstrap	4
2.2	Phương pháp K-fold Cross-Validation	6
3	Dữ liệu và tiền xử lý	9
3.1	Nguồn dữ liệu và cấu trúc	9
3.2	Thống kê mô tả và giá trị khuyết	10
4	Phân tích khám phá dữ liệu	11
4.1	Phân bố biến mục tiêu	11
4.2	Tương quan giữa các biến	11
4.3	Phân bố các biến sinh hoá và biến đổi log	12
4.4	Đa cộng tuyến và hệ số VIF	14
4.5	Thống kê mô tả theo nhóm	16
4.6	Lựa chọn đặc trưng	18
5	Thiết kế quy trình xây dựng mô hình	19
5.1	Tập biến đưa vào mô hình	19
5.2	Chia phân tầng (Stratified Sampling) và cân bằng lớp thiểu số bằng phương pháp SMOTE	19
5.3	Hồi quy logistic có điều chuẩn Ridge	21
5.4	Phòng ngừa rò rỉ dữ liệu trong quy trình huấn luyện	21
5.5	Bộ chỉ số đánh giá	22
6	Bootstrap và Kiểm định chéo phân tầng	23
6.1	Ước lượng khoảng tin cậy bằng Bootstrap	23
6.2	Kiểm định chéo phân tầng	25
7	Tinh chỉnh siêu tham số	27
8	Lựa chọn ngưỡng phân loại	28
9	Đánh giá trên tập kiểm tra	31
10	Kết luận	34
10.1	Hạn chế	35
10.2	Hướng mở rộng	35

1 Đặt vấn đề

Các bệnh lý về gan thường tiến triển âm thầm trong thời gian dài và chỉ biểu hiện triệu chứng rõ rệt khi bệnh đã bước sang giai đoạn nặng, làm giảm đáng kể hiệu quả của các biện pháp điều trị. Vì vậy, việc phát hiện sớm nguy cơ mắc bệnh thông qua các xét nghiệm sinh hóa thường quy, chẳng hạn như bilirubin, men gan và albumin, có ý nghĩa quan trọng trong hỗ trợ chẩn đoán và can thiệp kịp thời.

Bộ dữ liệu *Indian Liver Patient Dataset (ILPD)* gồm 583 mẫu dữ liệu, mỗi mẫu tương ứng với một bệnh nhân và bao gồm các chỉ số sinh hóa cùng nhãn chẩn đoán xác định. Từ đó, bài toán được đặt ra là xây dựng một mô hình phân loại nhị phân nhằm dự đoán tình trạng mắc bệnh gan dựa trên kết quả xét nghiệm. Đây là một bài toán điển hình trong học máy ứng dụng vào lĩnh vực y tế, nơi không chỉ yêu cầu mô hình đạt hiệu năng cao mà còn phải có khả năng tổng quát hóa tốt trên dữ liệu chưa từng được quan sát.

Trong thực tế, việc chỉ huấn luyện một mô hình và báo cáo một giá trị hiệu năng trên một tập dữ liệu là chưa đủ để khẳng định chất lượng của mô hình. Các chỉ số như Accuracy, F1-score hay ROC-AUC chỉ là những ước lượng thu được từ một mẫu dữ liệu hữu hạn và có thể thay đổi khi áp dụng trên những tập dữ liệu khác. Do đó, quá trình đánh giá cần giải quyết hai vấn đề quan trọng sau:

1. Trong số các mô hình và cấu hình siêu tham số được thử nghiệm, mô hình nào có khả năng tổng quát hóa tốt nhất? Để trả lời câu hỏi này, cần sử dụng một quy trình đánh giá khách quan nhằm ước lượng hiệu năng của mô hình trên dữ liệu chưa từng được sử dụng để huấn luyện ở mỗi lần đánh giá. *K-fold Cross-Validation* là phương pháp được sử dụng để đánh giá và hỗ trợ lựa chọn mô hình phù hợp.
2. Sau khi lựa chọn được mô hình, mức độ tin cậy của các chỉ số đánh giá là bao nhiêu? Thay vì chỉ báo cáo một giá trị ước lượng điểm (*point estimate*), cần ước lượng thêm khoảng tin cậy (*confidence interval*) để phản ánh mức độ biến thiên của các chỉ số đánh giá. Phương pháp *Bootstrap* được sử dụng nhằm ước lượng các khoảng tin cậy này.

Như vậy, *K-fold Cross-Validation* và *Bootstrap* đảm nhận hai vai trò bổ sung cho nhau trong quá trình đánh giá mô hình. Trong khi *K-fold Cross-Validation* được sử dụng để ước lượng khả năng tổng quát hóa và hỗ trợ lựa chọn mô hình, thì *Bootstrap* cho phép ước lượng khoảng tin cậy của các chỉ số đánh giá, từ đó phản ánh mức độ bất định của hiệu năng mô hình. Báo cáo này trình bày cơ sở lý thuyết của hai phương pháp, đồng thời xây dựng một quy trình học máy hoàn chỉnh trên bộ dữ liệu ILPD, bao gồm tiền xử lý dữ liệu, lựa chọn đặc trưng, tinh

chỉnh siêu tham số, tối ưu ngưỡng phân loại và ước lượng khoảng tin cậy cho hiệu năng của mô hình.

2 Cơ sở lý thuyết

Bootstrap và K-fold Cross-Validation đều thuộc nhóm các kỹ thuật *resampling* (lấy mẫu lại), được phát triển nhằm giải quyết một vấn đề cơ bản trong thống kê suy luận. Trong thực tế, nhà nghiên cứu chỉ có thể quan sát một mẫu hữu hạn được rút ra từ tổng thể, trong khi các đại lượng thống kê được tính từ mẫu đó, chẳng hạn như giá trị trung bình, hệ số hồi quy hay độ chính xác của mô hình, đều là những biến ngẫu nhiên. Nếu lấy một mẫu khác từ cùng tổng thể, các đại lượng này nhiều khả năng sẽ nhận những giá trị khác.

Do đó, câu hỏi quan trọng không chỉ là giá trị ước lượng bằng bao nhiêu, mà còn là mức độ biến thiên của ước lượng đó khi dữ liệu thay đổi. Trong nhiều trường hợp, phân phối lấy mẫu của đại lượng quan tâm không thể xác định chính xác hoặc quá phức tạp để suy ra bằng lý thuyết. Khi đó, các phương pháp *resampling* cho phép xấp xỉ phân phối này bằng cách tạo nhiều mẫu lấy lại từ dữ liệu ban đầu và quan sát sự thay đổi của đại lượng quan tâm trên các mẫu đó.

2.1 Phương pháp Bootstrap

Bootstrap là một phương pháp *resampling* do Bradley Efron đề xuất vào năm 1979, được sử dụng để ước lượng độ chính xác của các đại lượng thống kê, chẳng hạn như sai số chuẩn và khoảng tin cậy, mà không cần giả định trước về phân phối của tổng thể cũng như không yêu cầu thu thập thêm dữ liệu.

Nền tảng lý thuyết. Xét tham số tổng thể $\theta = \theta(F)$, trong đó F là hàm phân phối chưa biết của tổng thể. Từ mẫu quan sát $x_1, \dots, x_n$, ta thu được ước lượng $\hat{\theta} = \theta(\hat{F})$. Để đánh giá độ chính xác của $\hat{\theta}$, cần biết *phân phối lấy mẫu* của nó, tức phân phối của $\hat{\theta}$ nếu quá trình lấy mẫu kích thước n từ F được lặp lại vô số lần. Tuy nhiên, điều này không khả thi trong thực tế vì hàm phân phối F không được biết trước và thông thường chỉ có một mẫu quan sát duy nhất.

Ý tưởng cốt lõi của Bootstrap là thay thế tổng thể chưa biết F bằng *phân phối thực nghiệm* $\hat{F}$, tức phân phối rời rạc gán xác suất $1/n$ cho mỗi quan sát trong mẫu. Theo định lý Glivenko–Cantelli, $\hat{F}$ hội tụ đều về F khi kích thước mẫu tăng, do đó $\hat{F}$ có thể được xem là một xấp xỉ hợp lý của F . Khi đó, việc rút một mẫu kích thước n từ $\hat{F}$ tương đương với việc lấy mẫu ngẫu nhiên **có hoàn lại** từ dữ liệu gốc. Đây chính là nội dung của *nguyên lý plug-in*: sử dụng phân phối thực nghiệm để xấp xỉ phân phối của tổng thể, từ đó xấp xỉ phân phối lấy

mẫu của θ bằng phân phối của các ước lượng bootstrap $\hat{\theta}^*$ thu được từ các mẫu lấy lại.

Thuật toán.

1. Từ mẫu gốc có kích thước n , lấy mẫu ngẫu nhiên **có hoàn lại** cũng có kích thước n , thu được một mẫu bootstrap.
2. Tính đại lượng thống kê quan tâm $\hat{\theta}_b^*$ trên mẫu bootstrap đó.
3. Lặp lại hai bước trên B lần, với B thường từ khoảng 1 000 đến vài nghìn, thu được tập các ước lượng bootstrap $\hat{\theta}_1^*, \dots, \hat{\theta}_B^*$.
4. Sử dụng phân phối thực nghiệm của các ước lượng bootstrap để ước lượng sai số chuẩn và khoảng tin cậy của θ .

Sai số chuẩn và khoảng tin cậy. Sai số chuẩn bootstrap được ước lượng bằng độ lệch chuẩn của B giá trị thống kê bootstrap:

$$\widehat{SE}_{boot} = \sqrt{\frac{1}{B-1} \sum_{b=1}^B \left(\hat{\theta}_b^* - \bar{\theta}^* \right)^2}, \quad \bar{\theta}^* = \frac{1}{B} \sum_{b=1}^B \hat{\theta}_b^*.$$

Có nhiều phương pháp để xây dựng khoảng tin cậy từ các mẫu bootstrap, trong đó *phương pháp phân vị (Percentile Bootstrap)* là cách đơn giản và được sử dụng phổ biến. Với mức tin cậy $1 - \alpha$, hai đầu mút của khoảng tin cậy được xác định lần lượt bởi các phân vị mức $\alpha/2$ và $1 - \alpha/2$ của tập B giá trị bootstrap. Chẳng hạn, với mức tin cậy 95%, khoảng tin cậy được xác định bởi phân vị 2,5% và phân vị 97,5%.

Ưu điểm.

1. *Không yêu cầu giả định trước về phân phối của tổng thể.* Nhiều phương pháp xây dựng khoảng tin cậy cổ điển dựa trên giả định dữ liệu tuân theo phân phối chuẩn hoặc dựa vào các kết quả tiệm cận khi kích thước mẫu đủ lớn. Bootstrap không yêu cầu các giả định này, do đó đặc biệt hữu ích đối với dữ liệu có phân phối lệch hoặc trong những trường hợp khó áp dụng các công thức lý thuyết.
2. *Áp dụng được cho nhiều đại lượng thống kê.* Nhiều chỉ số đánh giá trong học máy, chẳng hạn như Recall, Precision hay Macro F1-score, không có công thức giải tích đơn giản để tính sai số chuẩn hoặc khoảng tin cậy. Bootstrap chỉ cần tính lại đại lượng quan tâm trên từng mẫu bootstrap, nhờ đó có thể áp dụng thống nhất cho hầu hết các thống kê.

3. *Không cần thu thập thêm dữ liệu.* Toàn bộ thông tin về mức độ biến thiên của ước lượng được khai thác từ chính mẫu quan sát ban đầu. Đây là một ưu điểm quan trọng trong các nghiên cứu y sinh, nơi việc mở rộng cỡ mẫu thường tốn kém cả về chi phí lẫn thời gian.
4. *Cho phép xấp xỉ phân phối lấy mẫu của thống kê.* Kết quả của Bootstrap không chỉ là sai số chuẩn hoặc khoảng tin cậy mà còn bao gồm toàn bộ tập các giá trị bootstrap. Nhờ đó, có thể khảo sát hình dạng của phân phối lấy mẫu xấp xỉ, đánh giá mức độ đối xứng, độ phân tán và tính ổn định của ước lượng.

Hạn chế.

1. *Chi phí tính toán cao.* Quá trình tính toán đại lượng thống kê phải được lặp lại hàng nghìn lần trên các mẫu bootstrap. Chi phí này càng tăng nếu mỗi lần lặp đều phải huấn luyện lại mô hình thay vì chỉ tính lại các chỉ số đánh giá từ những dự đoán đã có.
2. *Hiệu quả phụ thuộc vào kích thước và chất lượng mẫu.* Phân phối thực nghiệm $\hat{F}$ chỉ xấp xỉ tốt phân phối tổng thể F khi mẫu quan sát đủ lớn và có tính đại diện. Với cỡ mẫu nhỏ, các mẫu bootstrap chỉ được tạo từ một số lượng quan sát hạn chế, khiến phân phối bootstrap có thể chưa phản ánh chính xác phân phối lấy mẫu thực sự. Do đó, sai số chuẩn và khoảng tin cậy ước lượng có thể kém chính xác.
3. *Giả định các quan sát độc lập và cùng phân phối.* Bootstrap cơ bản giả định dữ liệu được lấy độc lập và cùng phân phối (i.i.d.). Vì vậy, phương pháp này không thể áp dụng trực tiếp cho các dữ liệu có cấu trúc phụ thuộc, chẳng hạn như chuỗi thời gian hoặc dữ liệu phân cụm theo bệnh viện hay khu vực địa lý.
4. *Không tạo ra thông tin mới.* Bootstrap chỉ khai thác lại thông tin từ mẫu quan sát ban đầu. Nếu bản thân mẫu gốc không đại diện cho tổng thể, chẳng hạn do sai lệch chọn mẫu, thì các khoảng tin cậy và ước lượng thu được vẫn có thể bị sai lệch so với giá trị thực của tổng thể.

2.2 Phương pháp K-fold Cross-Validation

K-fold Cross-Validation, hay kiểm định chéo k phần, là một kỹ thuật *resampling* **không hoàn lại**, được sử dụng để ước lượng khả năng tổng quát hóa của mô hình trên dữ liệu chưa từng được sử dụng trong quá trình huấn luyện. Nhờ đó, phương pháp giúp đánh giá khách quan hơn hiệu năng của mô hình so với việc đánh giá trực tiếp trên tập dữ liệu huấn luyện.

Nền tảng lý thuyết. Mục tiêu của một mô hình học máy là dự đoán tốt trên dữ liệu mới, thay vì chỉ khớp với dữ liệu đã quan sát. Nếu đánh giá mô hình trên chính tập dữ liệu dùng để huấn luyện, kết quả thu được thường quá lạc quan vì mô hình có thể đã học cả những dao động ngẫu nhiên đặc trưng của mẫu huấn luyện, hay còn gọi là hiện tượng *overfitting*. Một cách khắc phục đơn giản là tách riêng một tập kiểm tra độc lập. Tuy nhiên, phương pháp này làm giảm lượng dữ liệu dành cho huấn luyện và kết quả đánh giá phụ thuộc đáng kể vào một lần chia dữ liệu ngẫu nhiên, đặc biệt khi kích thước mẫu nhỏ. K-fold Cross-Validation khắc phục hạn chế này bằng cách luân phiên sử dụng các phần dữ liệu làm tập huấn luyện và tập kiểm tra, sao cho mỗi quan sát được sử dụng đúng một lần để kiểm tra và $k - 1$ lần để huấn luyện.

Thuật toán.

1. Chia ngẫu nhiên tập dữ liệu thành k phần rời nhau có kích thước xấp xỉ bằng nhau, gọi là các *fold*.
2. Với mỗi $i = 1, \dots, k$, sử dụng $k - 1$ fold để huấn luyện mô hình và fold còn lại làm tập kiểm tra.
3. Tính sai số dự đoán trên từng fold, sau đó lấy trung bình:

$$CV_{(k)} = \frac{1}{k} \sum_{i=1}^k Err_i,$$

trong đó Err_i là giá trị trung bình của hàm mất mát trên fold thứ i . Nếu ký hiệu $\mathcal{F}_i$ là tập các quan sát thuộc fold thứ i , ta có

$$Err_i = \frac{1}{|\mathcal{F}_i|} \sum_{j \in \mathcal{F}_i} L(y_j, \hat{f}^{(-i)}(x_j)),$$

với L là hàm mất mát và $\hat{f}^{(-i)}$ là mô hình được huấn luyện khi loại bỏ fold thứ i .

Ví dụ minh họa. Xét bài toán phân loại bệnh gan trên bộ dữ liệu ILPD gồm $n = 583$ quan sát bằng mô hình hồi quy logistic. Với $k = 5$, dữ liệu được chia thành 5 fold, mỗi fold gồm khoảng 117 quan sát. Ở mỗi lần lặp, mô hình được huấn luyện trên khoảng 466 quan sát thuộc 4 fold và được đánh giá trên fold còn lại. Quá trình được lặp lại 5 lần để mỗi fold lần lượt đóng vai trò tập kiểm tra. Giá trị trung bình của năm lần đánh giá được sử dụng để ước lượng hiệu năng dự đoán của mô hình, trong khi sự khác biệt giữa các fold phản ánh mức độ ổn định của mô hình đối với các cách chia dữ liệu khác nhau.

Biến thể Stratified K-fold. Khi biến mục tiêu bị mất cân bằng lớp, việc chia dữ liệu hoàn toàn ngẫu nhiên có thể tạo ra những fold có tỷ lệ lớp khác biệt

đáng kể so với toàn bộ dữ liệu, thậm chí không chứa lớp thiểu số. Điều này làm cho kết quả đánh giá có phương sai lớn và thiếu ổn định. **Stratified K-fold** khắc phục vấn đề này bằng cách duy trì tỷ lệ các lớp trong mỗi fold gần giống với tỷ lệ của toàn bộ tập dữ liệu. Đối với ILPD, tỷ lệ giữa nhóm mắc bệnh và không mắc bệnh xấp xỉ 71% và 29%, vì vậy mỗi fold đều được tạo sao cho phản ánh gần đúng tỷ lệ này. Nhờ đó, kết quả đánh giá giữa các fold ổn định và có tính đại diện cao hơn. Đây cũng là biến thể được sử dụng trong toàn bộ phần thực nghiệm của báo cáo.

Ưu điểm.

1. *Ước lượng khách quan hơn về khả năng tổng quát hóa.* Mỗi quan sát đều được dự đoán bởi một mô hình chưa từng sử dụng chính quan sát đó trong quá trình huấn luyện, nhờ đó giảm sự lạc quan của các chỉ số đánh giá.
2. *Sử dụng dữ liệu hiệu quả.* Toàn bộ các quan sát đều lần lượt tham gia vào cả quá trình huấn luyện và kiểm tra, giúp tận dụng tối đa dữ liệu, đặc biệt khi kích thước mẫu hạn chế.
3. *Đánh giá tính ổn định của mô hình.* Ngoài giá trị trung bình của các chỉ số đánh giá, K-fold Cross-Validation còn cung cấp kết quả trên từng fold, qua đó cho phép quan sát mức độ dao động của mô hình dưới các cách chia dữ liệu khác nhau.
4. *Áp dụng cho nhiều loại mô hình.* Phương pháp chỉ yêu cầu mô hình có khả năng huấn luyện và dự đoán, do đó có thể áp dụng thống nhất cho hầu hết các thuật toán học máy.

Hạn chế.

1. *Chi phí tính toán cao.* Mô hình phải được huấn luyện k lần thay vì một lần. Chi phí này còn tăng đáng kể khi K-fold Cross-Validation được sử dụng bên trong quá trình dò siêu tham số (Grid Search hoặc Random Search).
2. *Đánh đổi giữa bias và variance.* Khi k nhỏ, mỗi mô hình được huấn luyện trên ít dữ liệu hơn nên ước lượng sai số có xu hướng thiên lệch (bias) lớn hơn. Ngược lại, khi k tăng, các tập huấn luyện trở nên rất giống nhau, khiến phương sai của ước lượng Cross-Validation có xu hướng tăng.
3. *Các lần đánh giá không độc lập.* Các tập huấn luyện ở các fold khác nhau chia sẻ phần lớn các quan sát. Vì vậy, độ lệch chuẩn giữa các kết quả trên từng fold không phải là sai số chuẩn của ước lượng Cross-Validation mà chỉ nên được xem như một chỉ báo về mức độ ổn định của mô hình.

4. Có thể dẫn đến *selection bias* trong lựa chọn mô hình. Nếu cùng một kết quả K-fold Cross-Validation được sử dụng để so sánh rất nhiều mô hình hoặc cấu hình siêu tham số, mô hình được lựa chọn có thể vô tình phù hợp với cách chia fold cụ thể hơn là thật sự có khả năng tổng quát hóa tốt. Trong trường hợp này, nên sử dụng một tập kiểm tra độc lập hoặc Nested Cross-Validation để đánh giá cuối cùng.

3 Dữ liệu và tiền xử lý

3.1 Nguồn dữ liệu và cấu trúc

Bộ dữ liệu ILPD được thu thập tại vùng đông bắc bang Andhra Pradesh, Ấn Độ, gồm 583 hồ sơ bệnh nhân với 10 biến giải thích là tuổi, giới tính và tám chỉ số sinh hoá. Nhãn gốc mã hoá giá trị 1 cho trường hợp có bệnh và giá trị 2 cho trường hợp không bệnh; nhãn này được đổi về quy ước 1 và 0 để thuận tiện cho các công thức đếm ở phần sau.

```
col_names <- c(
  "Age", "Gender", "Total_Bilirubin", "Direct_Bilirubin",
  "Alkaline_Phosphotase", "Alamine_Aminotransferase",
  "Aspartate_Aminotransferase", "Total_Protiens", "Albumin",
  "Albumin_and_Globulin_Ratio", "target"
)

df <- read.csv("Indian Liver Patient Dataset (ILPD).csv",
  header = FALSE, col.names = col_names,
  stringsAsFactors = FALSE)

# Nhãn gốc: 1 = có bệnh, 2 = không bệnh. Đổi về 1 / 0.
df$target <- ifelse(df$target == 2, 0, 1)

cat("Kích thước dữ liệu:", nrow(df), "dòng x", ncol(df), "cột\n")

## Kích thước dữ liệu: 583 dòng x 11 cột

cat("Phân bố target:", sum(df$target == 1), "có bệnh /",
  sum(df$target == 0), "không bệnh\n")

## Phân bố target: 416 có bệnh / 167 không bệnh
```

```
str(df)

## 'data.frame': 583 obs. of 11 variables:
## $ Age : int 65 62 62 58 72 46 26 29 17 55 ...
## $ Gender : chr "Female" "Male" "Male" "Male" ...
## $ Total_Bilirubin : num 0.7 10.9 7.3 1 3.9 1.8 0.9 0.9 0.9 0.7 ...
## $ Direct_Bilirubin : num 0.1 5.5 4.1 0.4 2 0.7 0.2 0.3 0.3 0.2 ...
## $ Alkaline_Phosphotase : int 187 699 490 182 195 208 154 202 202 290 ...
## $ Alamine_Aminotransferase : int 16 64 60 14 27 19 16 14 22 53 ...
## $ Aspartate_Aminotransferase: int 18 100 68 20 59 14 12 11 19 58 ...
## $ Total_Protiens : num 6.8 7.5 7 6.8 7.3 7.6 7 6.7 7.4 6.8 ...
## $ Albumin : num 3.3 3.2 3.3 3.4 2.4 4.4 3.5 3.6 4.1 3.4 ...
## $ Albumin_and_Globulin_Ratio: num 0.9 0.74 0.89 1 0.4 1.3 1 1.1 1.2 1 ...
## $ target : num 1 1 1 1 1 1 1 1 0 1 ...
```

3.2 Thống kê mô tả và giá trị khuyết

```
summary(df)

##      Age      Gender  Total_Bilirubin  Direct_Bilirubin
## Min.   : 4.00   Length :583   Min.    : 0.400   Min.    : 0.100
## 1st Qu.:33.00   N.unique : 2   1st Qu.: 0.800   1st Qu.: 0.200
## Median :45.00   N.blank  : 0   Median : 1.000   Median : 0.300
## Mean   :44.75   Min.nchar: 4   Mean    : 3.299   Mean    : 1.486
## 3rd Qu.:58.00   Max.nchar: 6   3rd Qu.: 2.600   3rd Qu.: 1.300
## Max.   :90.00           Max.    :75.000   Max.    :19.700
##
## Alkaline_Phosphotase Alamine_Aminotransferase
## Min.    : 63.0      Min.    : 10.00
## 1st Qu.: 175.5      1st Qu.: 23.00
## Median : 208.0      Median : 35.00
## Mean    : 290.6      Mean    : 80.71
## 3rd Qu.: 298.0      3rd Qu.: 60.50
## Max.    :2110.0      Max.    :2000.00
##
## Aspartate_Aminotransferase Total_Protiens      Albumin
## Min.    : 10.0      Min.    :2.700   Min.    :0.900
## 1st Qu.: 25.0      1st Qu.:5.800   1st Qu.:2.600
## Median : 42.0      Median :6.600   Median :3.100
## Mean    :109.9      Mean    :6.483   Mean    :3.142
## 3rd Qu.: 87.0      3rd Qu.:7.200   3rd Qu.:3.800
## Max.    :4929.0      Max.    :9.600   Max.    :5.500
##
## Albumin_and_Globulin_Ratio      target
## Min.    :0.3000      Min.    :0.0000
## 1st Qu.:0.7000      1st Qu.:0.0000
## Median :0.9300      Median :1.0000
## Mean    :0.9471      Mean    :0.7136
## 3rd Qu.:1.1000      3rd Qu.:1.0000
## Max.    :2.8000      Max.    :1.0000
## NAs      :4

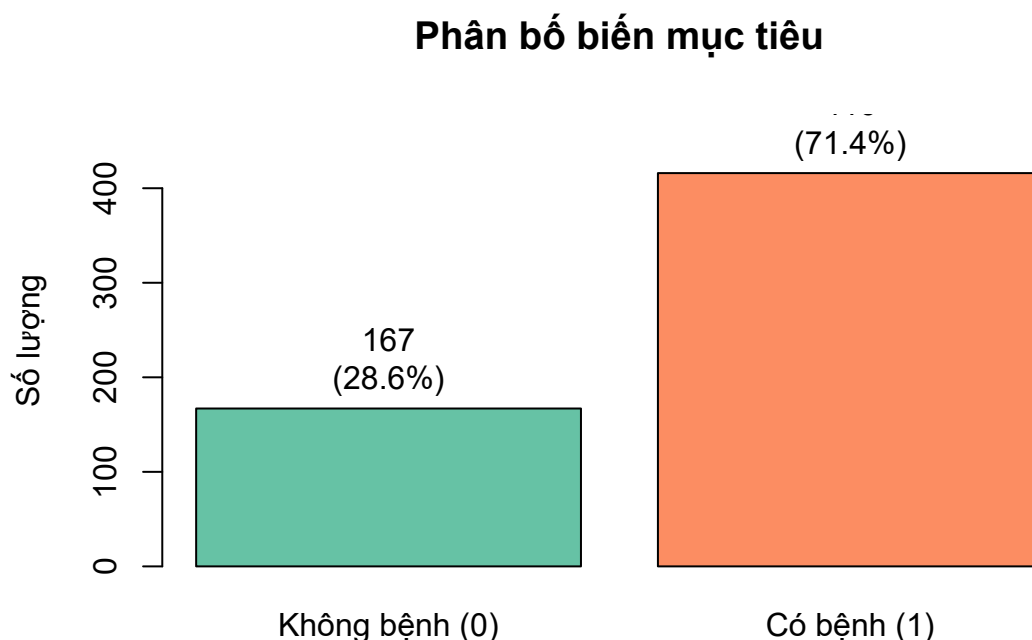
colSums(is.na(df))

##      Age      Gender
##      0      0
## Total_Bilirubin  Direct_Bilirubin
##      0      0
## Alkaline_Phosphotase Alamine_Aminotransferase
##      0      0
## Aspartate_Aminotransferase      Total_Protiens
##      0      0
## Albumin Albumin_and_Globulin_Ratio
##      0      4
## target
##      0
```

Toàn bộ dữ liệu chỉ có 4 giá trị khuyết, đều nằm ở biến `Albumin_and_Globulin_Ratio`. Con số này rất nhỏ so với 583 quan sát nên cách xử lý gần như không ảnh hưởng tới kết quả. Dù vậy, các giá trị khuyết vẫn được điền bằng trung vị thay vì loại bỏ quan sát, và điều quan trọng hơn là việc điền khuyết được đặt bên trong quy trình huấn luyện để trung vị chỉ được tính từ tập huấn luyện. Lý do của yêu cầu này được trình bày ở Mục 5.4.

4 Phân tích khám phá dữ liệu

4.1 Phân bố biến mục tiêu



Hình 1: Phân bố biến mục tiêu

Tỉ lệ 71,4% và 28,6% cho thấy dữ liệu mất cân bằng ở mức trung bình. Hệ quả trực tiếp là độ chính xác tổng thể trở thành một chỉ số gây hiểu nhầm, bởi một mô hình dự đoán tất cả bệnh nhân đều mắc bệnh đã đạt độ chính xác 0,714 mà không mang lại thông tin nào. Vì vậy toàn bộ phần đánh giá về sau sử dụng recall và precision của từng lớp cùng với macro F1, thay vì dựa vào độ chính xác tổng thể.

4.2 Tương quan giữa các biến

Ba cặp biến có tương quan rất cao. Cặp thứ nhất là Total_Bilirubin với Direct_Bilirubin ($r = 0,87$), cặp thứ hai là hai men gan Alanine_Aminotransferase và Aspartate_Aminotransferase ($r = 0,79$), cặp thứ ba là Total_Protiens với Albumin ($r = 0,78$). Các quan hệ này phù hợp với bản chất sinh hoá của những chỉ số đó, khi bilirubin trực tiếp là một thành phần của bilirubin toàn phần và albumin là thành phần chính của protein toàn phần. Tuy nhiên chúng lại gây khó khăn cho hồi quy logistic, bởi các biến

Ma trận tương quan

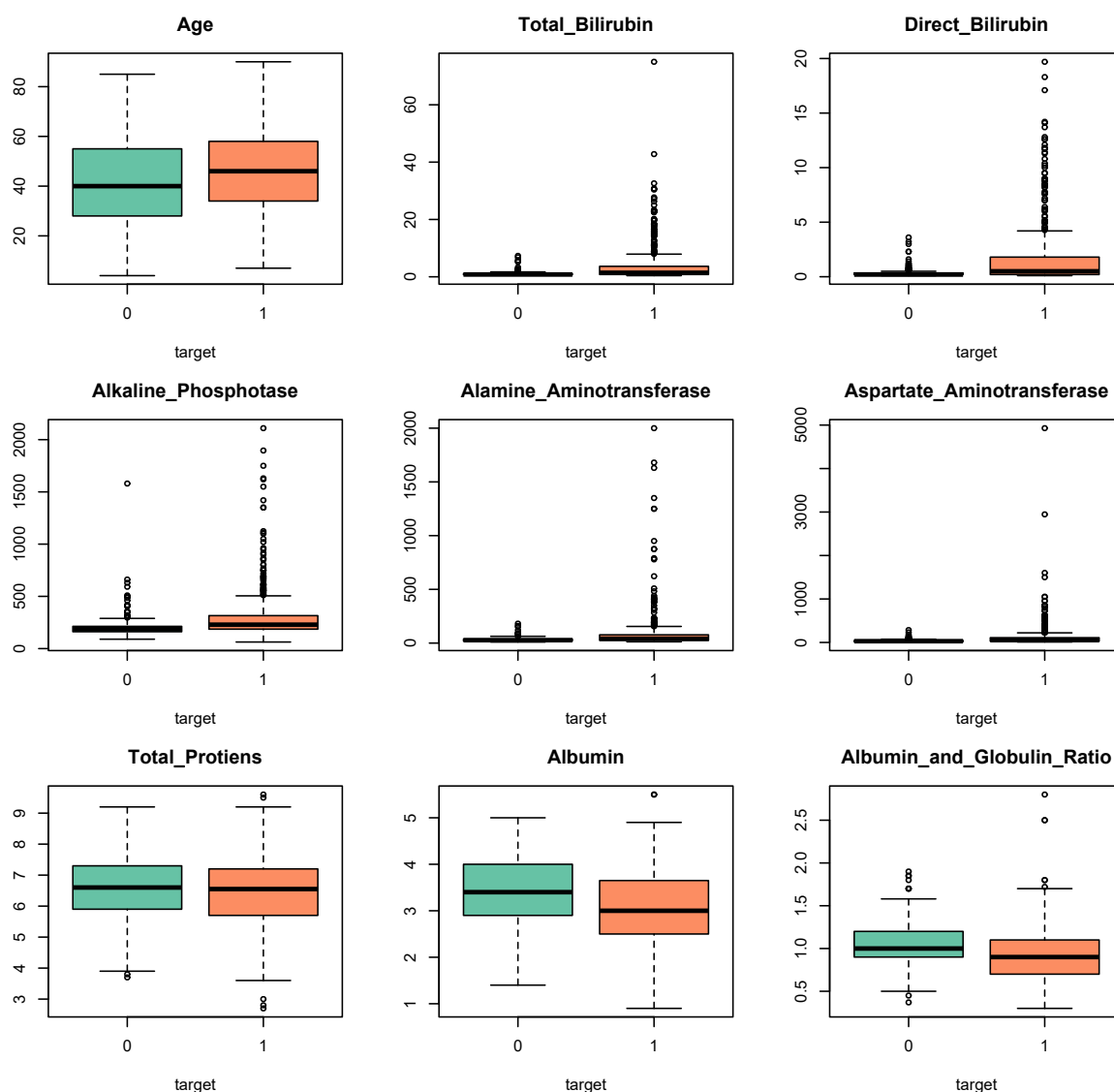
Age	1.00	0.01	0.01	0.08	-0.09	-0.02	-0.19	-0.26	-0.22	0.13
Total_Bilirubin	0.01	1.00	0.87	0.21	0.21	0.24	-0.01	-0.22	-0.21	0.22
Direct_Bilirubin	0.01	0.87	1.00	0.23	0.23	0.26	0.00	-0.23	-0.20	0.25
Alkaline_Phosphotase	0.08	0.21	0.23	1.00	0.12	0.17	-0.03	-0.16	-0.23	0.18
Alamine_Aminotransferase	-0.09	0.21	0.23	0.12	1.00	0.79	-0.04	-0.03	-0.00	0.16
Aspartate_Aminotransferase	-0.02	0.24	0.26	0.17	0.79	1.00	-0.03	-0.08	-0.07	0.15
Total_Protiens	-0.19	-0.01	0.00	-0.03	-0.04	-0.03	1.00	0.78	0.23	-0.03
Albumin	-0.26	-0.22	-0.23	-0.16	-0.03	-0.08	0.78	1.00	0.69	-0.16
Albumin_and_Globulin_Ratio	-0.22	-0.21	-0.20	-0.23	-0.00	-0.07	0.23	0.69	1.00	-0.16
target	0.13	0.22	0.25	0.18	0.16	0.15	-0.03	-0.16	-0.16	1.00
	Age	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotransferase	Aspartate_Aminotransferase	Total_Protiens	Albumin	Albumin_and_Globulin_Ratio	target

Hình 2: Ma trận tương quan giữa các biến sinh hoá và biến mục tiêu

gần như trùng thông tin sẽ làm ma trận thiết kế gần suy biến và khiến hệ số ước lượng mất ổn định.

Ở chiều ngược lại, tương quan giữa từng biến với biến mục tiêu đều yếu, giá trị lớn nhất chỉ đạt 0,25. Điều này cho thấy không có biến đơn lẻ nào tách được hai nhóm một cách rõ ràng.

4.3 Phân bố các biến sinh hoá và biến đổi log



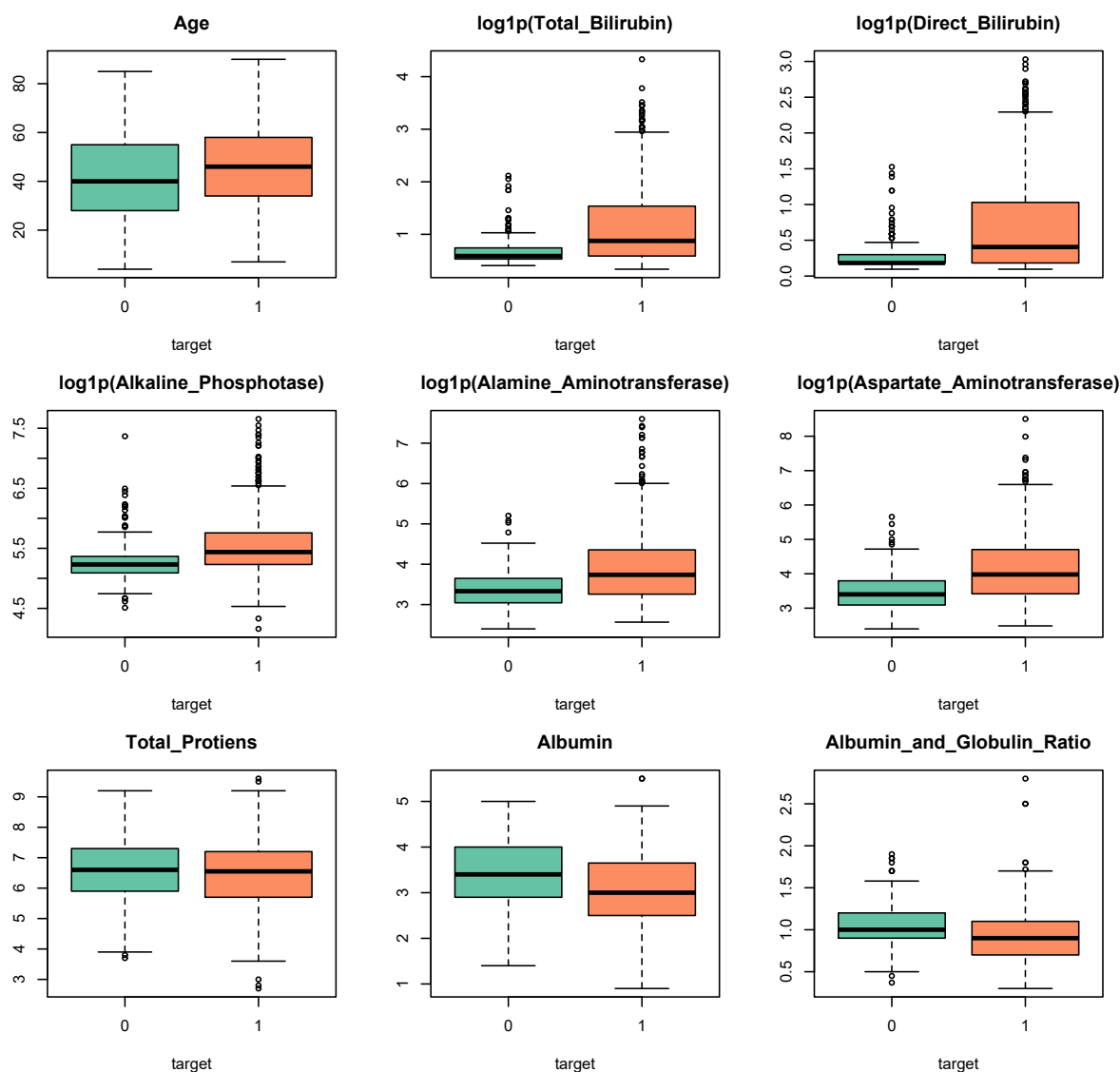
Hình 3: Phân bố các biến sinh hoá theo nhóm bệnh trên thang đo gốc

Hình 3 cho thấy năm biến gồm Total_Bilirubin, Direct_Bilirubin, Alkaline_Phosphotase, Alamine_Aminotransferase và Aspartate_Aminotransferase đều có phân phối lệch phải rõ rệt. Phần lớn các quan sát tập trung ở vùng giá trị thấp, trong khi một số ít giá trị rất lớn tạo thành đuôi phân phối kéo dài và nhiều điểm ngoại lai. Do đó, phần hộp của boxplot bị nén sát phía dưới, gây khó khăn cho việc quan sát và so sánh phân bố giữa hai nhóm bệnh.

Để giảm mức độ bất đối xứng, phép biến đổi $\log(1 + x)$ được áp dụng cho năm biến trên. Vì $\log(1 + x)$ là một hàm đơn điệu tăng nên phép biến đổi này bảo toàn thứ tự giữa các quan sát, đồng thời làm giảm ảnh hưởng của các giá trị lớn bằng cách nén phần đuôi của phân phối.

Hiệu quả của phép biến đổi được thể hiện ở Hình 4. Sau khi biến đổi log, các phân phối trở nên cân đối hơn, phần hộp của boxplot được trải rộng thay vì

tập trung sát đáy, trong khi các giá trị ngoại lai không còn chi phối mạnh thang đo của biểu đồ. Nhờ đó, sự khác biệt về vị trí trung tâm và mức độ phân tán giữa hai nhóm bệnh được thể hiện trực quan hơn, tạo điều kiện thuận lợi cho các bước phân tích và xây dựng mô hình ở các phần tiếp theo.



Hình 4: Phân bố các biến sinh hoá theo nhóm bệnh sau biến đổi log

4.4 Đa cộng tuyến và hệ số VIF

Hệ số tương quan chỉ phản ánh mối quan hệ giữa từng cặp biến, trong khi hiện tượng đa cộng tuyến có thể phát sinh từ mối quan hệ đồng thời giữa một biến với nhiều biến giải thích khác. Vì vậy, báo cáo sử dụng hệ số phóng đại phương sai (Variance Inflation Factor – VIF) để đánh giá mức độ đa cộng tuyến của từng

biến. Hệ số VIF được xác định theo công thức

$$\text{VIF}_j = \frac{1}{1 - R_j^2},$$

trong đó R_j^2 là hệ số xác định của mô hình hồi quy biến thứ j theo tất cả các biến còn lại. Theo quy ước thường được sử dụng, giá trị VIF lớn hơn 5 cho thấy dấu hiệu đa cộng tuyến đáng lưu ý, trong khi VIF lớn hơn 10 được xem là nghiêm trọng.

Khác với các kiểm định dựa trên thứ hạng, hệ số VIF không bất biến đối với các phép biến đổi đơn điệu như phép biến đổi log. Do mô hình trong nghiên cứu được xây dựng trên dữ liệu sau biến đổi log, VIF được tính trên cả thang đo gốc và thang đo log nhằm đánh giá ảnh hưởng của phép biến đổi đến mức độ đa cộng tuyến.

```
compute_vif <- function(data) {  
  vars <- colnames(data)  
  sapply(vars, function(v) {  
    others <- setdiff(vars, v)  
    fml <- as.formula(paste(v, "~", paste(others, collapse = " + ")))  
    r2 <- summary(lm(fml, data = data))$r.squared  
    1 / (1 - r2)  
  })  
}  
  
# Thang gốc: dùng df_raw (bản trước biến đổi Log)  
X_vif <- df_raw[, c(numeric_cols, "Gender")]  
X_vif$Gender <- ifelse(X_vif$Gender == "Male", 1, 0)  
X_vif$Albumin_and_Globulin_Ratio[is.na(X_vif$Albumin_and_Globulin_Ratio)] <-  
  median(X_vif$Albumin_and_Globulin_Ratio, na.rm = TRUE)  
vif_raw <- compute_vif(X_vif)  
  
# Thang log: log1p cho các biến lệch phải  
X_vif_log <- X_vif  
for (col in skewed_cols) X_vif_log[[col]] <- log1p(df_raw[[col]])  
vif_log <- compute_vif(X_vif_log)  
  
vif_tbl <- data.frame(  
  Bien = names(vif_raw),  
  VIF_goc = round(vif_raw, 2),  
  VIF_log = round(vif_log[names(vif_raw)], 2),  
  chenh_lech = round(vif_log[names(vif_raw)] - vif_raw, 2),  
  row.names = NULL  
)  
vif_tbl <- vif_tbl[order(-vif_tbl$VIF_log), ]
```

Bảng 1: Hệ số VIF trên thang đo gốc và thang đo log

Biến	VIF gốc	VIF log	Chênh lệch
Total_Bilirubin	4.28	20.72	16.44
Direct_Bilirubin	4.53	20.55	16.03
Albumin	10.12	10.62	0.49
Total_Protiens	5.55	5.65	0.10
Aspartate_Aminotransferase	2.81	4.18	1.37
Alamine_Aminotransferase	2.82	3.94	1.12
Albumin_and_Globulin_Ratio	3.68	3.75	0.07
Alkaline_Phosphotase	1.12	1.28	0.16
Age	1.10	1.10	0.00
Gender	1.03	1.06	0.03

Kết quả cho thấy phép biến đổi log ảnh hưởng đáng kể đến mức độ đa cộng tuyến của một số biến. Đáng chú ý nhất là hai biến Total_Bilirubin và Direct_Bilirubin, với giá trị VIF tăng từ khoảng 4,3 trên thang đo gốc lên trên 20 sau khi biến đổi log. Điều này cho thấy mối quan hệ tuyến tính giữa hai biến trở nên mạnh hơn trên thang đo log, dẫn đến hiện tượng đa cộng tuyến nghiêm trọng.

Đối với các biến còn lại, giá trị VIF thay đổi không đáng kể sau phép biến đổi. Riêng biến Albumin có VIF xấp xỉ 10 trên cả hai thang đo, cho thấy hiện tượng đa cộng tuyến đã tồn tại ngay từ dữ liệu gốc và hầu như không chịu ảnh hưởng của phép biến đổi log. Các biến khác đều có VIF nhỏ hơn 6, cho thấy mức độ đa cộng tuyến không đáng kể.

Do mô hình được xây dựng trên dữ liệu sau biến đổi log, kết quả VIF trên thang đo này được sử dụng để lựa chọn đặc trưng. Trong đó, Direct_Bilirubin được loại bỏ do có mức đa cộng tuyến rất cao với Total_Bilirubin, trong khi Total_Bilirubin được giữ lại vì mang ý nghĩa lâm sàng tổng quát hơn.

4.5 Thống kê mô tả theo nhóm

Bảng thống kê dưới đây được tính trên thang đo gốc, để các con số giữ nguyên đơn vị lâm sàng và có thể đối chiếu với khoảng tham chiếu xét nghiệm.

```
group_stats <- do.call(rbind, lapply(numeric_cols, function(col) {
  s <- tapply(df_raw[[col]], df_raw$target, function(x) {
    x <- x[!is.na(x)]
    c(Min = min(x), Max = max(x), Mean = mean(x),
      Median = median(x), SD = sd(x))
  })
  data.frame(Bien = col, Nhóm = c("Không bệnh (0)", "Có bệnh (1)"),
    round(do.call(rbind, s), 2), row.names = NULL)
}))
```

Bảng 2: Thống kê mô tả các biến sinh hoá theo nhóm bệnh (thang đo gốc)

Biến	Nhóm	Min	Max	Mean	Median	SD
Age	Không bệnh (0)	4.00	85.0	41.24	40.00	17.00
Age	Có bệnh (1)	7.00	90.0	46.15	46.00	15.65
Total_Bilirubin	Không bệnh (0)	0.50	7.3	1.14	0.80	1.00
Total_Bilirubin	Có bệnh (1)	0.40	75.0	4.16	1.40	7.14
Direct_Bilirubin	Không bệnh (0)	0.10	3.6	0.40	0.20	0.52
Direct_Bilirubin	Có bệnh (1)	0.10	19.7	1.92	0.50	3.21
Alkaline_Phosphotase	Không bệnh (0)	90.00	1580.0	219.75	186.00	140.99
Alkaline_Phosphotase	Có bệnh (1)	63.00	2110.0	319.01	229.00	268.31
Alamine_Aminotransferase	Không bệnh (0)	10.00	181.0	33.65	27.00	25.06
Alamine_Aminotransferase	Có bệnh (1)	12.00	2000.0	99.61	41.00	212.77
Aspartate_Aminotransferase	Không bệnh (0)	10.00	285.0	40.69	29.00	36.41
Aspartate_Aminotransferase	Có bệnh (1)	11.00	4929.0	137.70	52.50	337.39
Total_Protiens	Không bệnh (0)	3.70	9.2	6.54	6.60	1.06
Total_Protiens	Có bệnh (1)	2.70	9.6	6.46	6.55	1.09
Albumin	Không bệnh (0)	1.40	5.0	3.34	3.40	0.78
Albumin	Có bệnh (1)	0.90	5.5	3.06	3.00	0.79
Albumin_and_Globulin_Ratio	Không bệnh (0)	0.37	1.9	1.03	1.00	0.29
Albumin_and_Globulin_Ratio	Có bệnh (1)	0.30	2.8	0.91	0.90	0.33

Bảng thống kê mô tả cho thấy nhóm bệnh nhân mắc bệnh gan (target = 1) có xu hướng lớn tuổi hơn so với nhóm không mắc bệnh, với tuổi trung bình lần lượt là 46,15 và 41,24.

Đối với các chỉ số bilirubin và men gan, nhóm mắc bệnh có giá trị trung bình, trung vị và độ phân tán đều cao hơn rõ rệt. Cụ thể, Total_Bilirubin tăng từ 1,14 lên 4,16 mg/dL, Direct_Bilirubin tăng từ 0,40 lên 1,92 mg/dL, trong khi các men gan Alamine_Aminotransferase, Aspartate_Aminotransferase và Alkaline_Phosphotase đều có giá trị trung bình cao hơn đáng kể ở nhóm mắc bệnh. Đồng thời, độ lệch chuẩn của các biến này cũng lớn hơn nhiều, cho thấy mức độ biến thiên cao và sự xuất hiện của các giá trị rất lớn trong nhóm bệnh.

Ngược lại, các chỉ số phản ánh chức năng tổng hợp của gan có xu hướng giảm ở nhóm mắc bệnh. Giá trị trung bình của Albumin giảm từ 3,34 xuống 3,06 g/dL, trong khi tỷ số Albumin_and_Globulin_Ratio giảm từ 1,03 xuống 0,91. Hai nhóm có giá trị Total_Proteins tương đối tương đồng, với trung bình lần lượt là 6,54 và 6,46 g/dL.

Nhìn chung, các thống kê mô tả cho thấy nhóm mắc bệnh có xu hướng tăng các chỉ số bilirubin và men gan, đồng thời giảm nồng độ albumin và tỷ số albumin/globulin.

4.6 Lựa chọn đặc trưng

Mục tiêu của bước này là xác định những đặc trưng có mối liên hệ với biến mục tiêu - tức tình trạng mắc bệnh gan. Quá trình thực hiện kiểm định giả thuyết được xem như một bước sàng lọc ban đầu nhằm loại bỏ các biến không có bằng chứng thống kê về mối liên hệ với biến mục tiêu. Ta tiến hành kiểm định với cặp giả thuyết sau:

- H_0 : Biến không có mối liên hệ với tình trạng mắc bệnh gan (phân phối của biến là như nhau giữa hai nhóm đối với biến liên tục, hoặc độc lập với biến mục tiêu đối với biến phân loại).
- H_1 : Biến có mối liên hệ với tình trạng mắc bệnh gan.

Do hầu hết các biến liên tục có phân phối lệch, sự khác biệt giữa hai nhóm được đánh giá bằng kiểm định phi tham số Mann–Whitney U. Riêng biến Gender là biến phân loại nên được kiểm định bằng phép kiểm Chi-square. Để kiểm soát sai số do thực hiện nhiều phép kiểm đồng thời, các giá trị p được hiệu chỉnh bằng thủ tục Benjamini–Hochberg nhằm kiểm soát tỷ lệ phát hiện sai (False Discovery Rate – FDR).

```
fs <- data.frame(feature = character(), p_value = numeric(),
                 effect_size = numeric(), stringsAsFactors = FALSE)

for (col in numeric_cols) {
  g0 <- df[[col]][df$target == 0]
  g1 <- df[[col]][df$target == 1]
  w <- wilcox.test(g0, g1, exact = FALSE)
  n0 <- sum(!is.na(g0)); n1 <- sum(!is.na(g1))
  rb <- 1 - 2 * w$statistic / (n0 * n1) # rank-biserial correlation
  fs <- rbind(fs, data.frame(feature = col, p_value = w$p.value,
                             effect_size = as.numeric(rb)))
}

# Gender là biến phân loại: dùng Chi-square, kích thước ảnh hưởng Cramer's V
contingency <- table(df$Gender, df$target)
chi_res <- chisq.test(contingency)
n_total <- sum(contingency)
cramers_v <- sqrt(chi_res$statistic /
                  (n_total * (min(dim(contingency)) - 1)))
fs <- rbind(fs, data.frame(feature = "Gender", p_value = chi_res$p.value,
                           effect_size = as.numeric(cramers_v)))

fs$p_fdr <- p.adjust(fs$p_value, method = "BH") # Benjamini-Hochberg
fs$significant <- fs$p_fdr < 0.05
fs <- fs[order(fs$p_value), ]

fs_disp <- fs
fs_disp$p_value <- signif(fs_disp$p_value, 3)
fs_disp$effect_size <- round(fs_disp$effect_size, 3)
fs_disp$p_fdr <- signif(fs_disp$p_fdr, 3)
```

Bảng 3: Kiểm định lựa chọn đặc trưng (Mann–Whitney U / Chi-square, hiệu chỉnh FDR)

Biến	p-value	Kích thước ảnh hưởng	p (FDR)	Có ý nghĩa
Aspartate_Aminotransferase	0.00e+00	0.394	0.00e+00	TRUE
Total_Bilirubin	0.00e+00	0.387	0.00e+00	TRUE
Direct_Bilirubin	0.00e+00	0.372	0.00e+00	TRUE
Alamine_Aminotransferase	0.00e+00	0.371	0.00e+00	TRUE
Alkaline_Phosphotase	0.00e+00	0.349	0.00e+00	TRUE
Albumin_and_Globulin_Ratio	5.80e-06	-0.240	9.70e-06	TRUE
Albumin	5.57e-05	-0.213	7.95e-05	TRUE
Age	1.77e-03	0.165	2.22e-03	TRUE
Gender	5.97e-02	0.078	6.63e-02	FALSE
Total_Protiens	4.37e-01	-0.041	4.37e-01	FALSE

Kết quả cho thấy 8 trong số 10 biến vẫn đạt ý nghĩa thống kê sau hiệu chỉnh FDR. Hai biến không đạt mức ý nghĩa là Gender ($p_{FDR} = 0,066$) và Total_Proteins ($p_{FDR} = 0,437$), do đó được loại khỏi mô hình. Trong các biến còn lại, nhóm chỉ số bilirubin và men gan có kích thước ảnh hưởng lớn nhất (effect size khoảng 0,35–0,39), cho thấy khả năng phân biệt tốt giữa hai nhóm bệnh.

5 Thiết kế quy trình xây dựng mô hình

5.1 Tập biến đưa vào mô hình

Từ 10 biến ban đầu, ba biến được loại bỏ theo các tiêu chí lựa chọn đặc trưng đã trình bày. Biến Direct_Bilirubin bị loại do hiện tượng đa cộng tuyến nghiêm trọng với Total_Bilirubin, thể hiện qua hệ số tương quan khoảng 0,87 và giá trị VIF trên thang log vượt 20. Hai biến Gender và Total_Proteins không đạt ý nghĩa thống kê sau khi hiệu chỉnh FDR, nên không được đưa vào mô hình.

```
model_cols <- c("Age", "Total_Bilirubin", "Alkaline_Phosphotase",
               "Alamine_Aminotransferase", "Aspartate_Aminotransferase",
               "Albumin", "Albumin_and_Globulin_Ratio")

X_all <- as.matrix(df[, model_cols])
y_all <- df$target
```

5.2 Chia phân tầng (Stratified Sampling) và cân bằng lớp thiểu số bằng phương pháp SMOTE

Bộ dữ liệu ILPD mất cân bằng lớp, với khoảng 71% bệnh nhân mắc bệnh và 29% không mắc bệnh. Vì vậy, báo cáo áp dụng hai kỹ thuật xử lý ở hai giai

đoạn khác nhau: chia phân tầng để duy trì tỷ lệ lớp trong các tập dữ liệu và SMOTE để cân bằng lớp trong quá trình huấn luyện.

Đối với bước chia dữ liệu, các quan sát trong từng lớp được xáo trộn ngẫu nhiên rồi chia riêng trước khi ghép lại, nhờ đó tỷ lệ giữa hai lớp được duy trì xấp xỉ trong tập huấn luyện, tập kiểm tra và các fold của Cross-Validation.

```
stratified_split <- function(y, test_frac = 0.2, seed = 42) {
  set.seed(seed)
  test_idx <- integer(0)
  for (cls in sort(unique(y))) {
    idx <- which(y == cls)
    test_idx <- c(test_idx, sample(idx, round(length(idx) * test_frac)))
  }
  list(train = setdiff(seq_along(y), test_idx), test = sort(test_idx))
}

stratified_folds <- function(y, k = 5, seed = 42) {
  set.seed(seed)
  fold <- integer(length(y))
  for (cls in sort(unique(y))) {
    idx <- sample(which(y == cls))
    fold[idx] <- rep_len(seq_len(k), length(idx))
  }
  fold
}
```

Để giảm ảnh hưởng của sự mất cân bằng trong quá trình huấn luyện, báo cáo sử dụng *Synthetic Minority Over-sampling Technique* (SMOTE). Thay vì sao chép các quan sát hiện có, SMOTE sinh các mẫu tổng hợp bằng phép nội suy tuyến tính giữa một quan sát của lớp thiểu số và một trong các láng giềng gần nhất cùng lớp:

$$x_{new} = x_a + u \cdot (x_b - x_a), \quad u \sim \mathcal{U}(0, 1).$$

```
smote_oversample <- function(X, y, k = 5, seed = 42) {
  set.seed(seed)
  tab <- table(y)
  min_lab <- as.numeric(names(tab)[which.min(tab)])
  n_new <- as.integer(max(tab) - min(tab))
  if (n_new <= 0) return(list(X = X, y = y))

  X_min <- X[y == min_lab, , drop = FALSE]
  if (nrow(X_min) <= k) k <- max(1, nrow(X_min) - 1)

  # Khoảng cách trong nội bộ lớp thiểu số; chặn đường chéo để không tự chọn mình
  D <- as.matrix(dist(X_min))
  diag(D) <- Inf

  synth <- matrix(0, nrow = n_new, ncol = ncol(X))
  for (i in seq_len(n_new)) {
    a <- sample.int(nrow(X_min), 1)
    neighbours <- order(D[a, ])[seq_len(k)]
    b <- neighbours[sample.int(k, 1)]
    gap <- runif(1)
    synth[i, ] <- X_min[a, ] + gap * (X_min[b, ] - X_min[a, ])
  }
  colnames(synth) <- colnames(X)
  list(X = rbind(X, synth), y = c(y, rep(min_lab, n_new)))
}
```

5.3 Hồi quy logistic có điều chuẩn Ridge

Mô hình phân loại được sử dụng là hồi quy logistic có bổ sung phạt L_2 , còn gọi là điều chuẩn Ridge. Mô hình này phù hợp với biến mục tiêu nhị phân, cho phép dự đoán xác suất mắc bệnh, đồng thời đủ gọn để có thể huấn luyện lặp lại nhiều lần trong các bước bootstrap và kiểm định chéo.

Thành phần phạt Ridge làm co các hệ số hồi quy về gần 0, nhờ đó giúp mô hình ổn định hơn khi các biến giải thích còn tương quan với nhau. Cường độ co được điều khiển bởi tham số λ ; λ càng lớn thì mô hình càng bị điều chuẩn mạnh và xác suất dự đoán có xu hướng tập trung hơn quanh 0,5. Trong báo cáo này, hồi quy logistic Ridge chỉ đóng vai trò mô hình nền. Tham số λ sẽ được dò cùng với số láng giềng của SMOTE ở phần tinh chỉnh siêu tham số, còn trọng tâm phân tích vẫn là quy trình đánh giá bằng bootstrap và kiểm định chéo.

```
fit_logreg_ride <- function(X, y, lambda = 0, max_iter = 50, tol = 1e-8) {  
  Xd <- cbind(1, X) # thêm cột hệ số chặn  
  p_dim <- ncol(Xd)  
  beta <- rep(0, p_dim)  
  
  P <- diag(lambda, p_dim)  
  P[1, 1] <- 0 # không điều chuẩn hệ số chặn  
  
  for (it in seq_len(max_iter)) {  
    eta <- as.vector(Xd %*% beta)  
    mu <- 1 / (1 + exp(-eta)) # xác suất dự đoán  
    W <- mu * (1 - mu) # trọng số IRLS  
    W[W < 1e-10] <- 1e-10 # chặn dưới để ma trận không suy biến  
    z <- eta + (y - mu) / W # biến phân hồi hiệu chỉnh  
  
    beta_new <- tryCatch(  
      solve(t(Xd) %*% (Xd * W) + P, t(Xd) %*% (W * z)),  
      error = function(e) beta  
    )  
    if (max(abs(beta_new - beta)) < tol) { beta <- beta_new; break }  
    beta <- as.vector(beta_new)  
  }  
  as.vector(beta)  
}  
  
predict_proba <- function(beta, X) {  
  1 / (1 + exp(-as.vector(cbind(1, X) %*% beta)))  
}
```

5.4 Phòng ngừa rò rỉ dữ liệu trong quy trình huấn luyện

Để tránh rò rỉ dữ liệu trong quá trình đánh giá mô hình, toàn bộ quy trình huấn luyện được thực hiện độc lập trong từng fold của phép kiểm định chéo, bao gồm bốn bước:

1. Điền khuyết giá trị thiếu bằng trung vị;
2. Chuẩn hoá dữ liệu;
3. Cân bằng lớp bằng SMOTE;

4. Huấn luyện mô hình hồi quy logistic Ridge.

Ba bước đầu đều cần ước lượng tham số từ dữ liệu huấn luyện. Nếu các bước này được thực hiện trước khi chia fold, thông tin từ tập kiểm tra sẽ vô tình được sử dụng trong quá trình huấn luyện. Đối với điền khuyết và chuẩn hoá, các tham số như trung vị, trung bình và độ lệch chuẩn sẽ bị ảnh hưởng bởi dữ liệu kiểm tra. Đối với SMOTE, các mẫu tổng hợp có thể được sinh từ những quan sát thuộc tập kiểm tra, khiến thông tin của tập kiểm tra xuất hiện trong tập huấn luyện.

Vì vậy, ở mỗi fold, toàn bộ quy trình tiền xử lý và huấn luyện được thực hiện chỉ trên tập huấn luyện. Các tham số của bước tiền xử lý sau đó được lưu lại để áp dụng nguyên vẹn cho dữ liệu mới.

```
fit_pipeline <- function(X_tr, y_tr, lambda = 1, smote_k = 5, seed = 42) {  
  # 1. Điền khuyết bằng median CỦA TẬP TRAIN  
  med <- apply(X_tr, 2, median, na.rm = TRUE)  
  for (j in seq_len(ncol(X_tr))) X_tr[is.na(X_tr[, j]), j] <- med[j]  
  
  # 2. Chuẩn hoá theo mean/sd CỦA TẬP TRAIN  
  mu <- colMeans(X_tr)  
  sdv <- apply(X_tr, 2, sd)  
  sdv[sdv == 0] <- 1  
  X_tr_s <- scale(X_tr, center = mu, scale = sdv)  
  
  # 3. SMOTE chỉ trên tập train  
  res <- smote_oversample(X_tr_s, y_tr, k = smote_k, seed = seed)  
  
  # 4. Khớp mô hình  
  beta <- fit_logreg_ridge(res$X, res$y, lambda = lambda)  
  
  list(beta = beta, median = med, mean = mu, sd = sdv)  
}  
  
# Áp dụng chuỗi biến đổi đã học từ train lên dữ liệu mới  
pipeline_proba <- function(fit, X_new) {  
  for (j in seq_len(ncol(X_new))) X_new[is.na(X_new[, j]), j] <- fit$median[j]  
  X_s <- scale(X_new, center = fit$mean, scale = fit$sd)  
  predict_proba(fit$beta, X_s)  
}
```

Khi dự đoán, dữ liệu mới chỉ được điền khuyết và chuẩn hoá bằng các tham số đã ước lượng từ tập huấn luyện. SMOTE không được áp dụng ở bước dự đoán vì đây là kỹ thuật chỉ dùng để cân bằng dữ liệu trong giai đoạn huấn luyện.

5.5 Bộ chỉ số đánh giá

Như đã nêu khi phân tích phân bố biến mục tiêu, độ chính xác tổng thể không phản ánh đúng chất lượng mô hình trên dữ liệu mất cân bằng. Bộ chỉ số dưới đây vì vậy tách riêng recall và precision cho từng lớp, tất cả đều tính trực tiếp từ bốn thành phần của ma trận nhầm lẫn.

Hai loại sai sót cũng không tương đương nhau về hậu quả lâm sàng. Âm tính giả (FN) là bệnh nhân có bệnh nhưng bị bỏ sót, dẫn tới chậm trễ trong chẩn đoán và điều trị. Dương tính giả (FP) là người không bệnh bị dự đoán nhầm,

hậu quả chủ yếu là phải làm thêm xét nghiệm xác nhận. Do đó recall của lớp có bệnh, tức tỉ lệ bệnh nhân thực sự được phát hiện, là chỉ số được quan tâm nhiều nhất trong bài toán này.

```
compute_metrics <- function(y_true, y_pred) {
  tp <- sum(y_true == 1 & y_pred == 1)
  tn <- sum(y_true == 0 & y_pred == 0)
  fp <- sum(y_true == 0 & y_pred == 1)
  fn <- sum(y_true == 1 & y_pred == 0)

  recall1 <- if ((tp + fn) > 0) tp / (tp + fn) else NA # độ nhạy
  recall0 <- if ((tn + fp) > 0) tn / (tn + fp) else NA # độ đặc hiệu
  prec1 <- if ((tp + fp) > 0) tp / (tp + fp) else 0
  prec0 <- if ((tn + fn) > 0) tn / (tn + fn) else 0
  f1_1 <- if ((prec1 + recall1) > 0) 2*prec1*recall1/(prec1+recall1) else 0
  f1_0 <- if ((prec0 + recall0) > 0) 2*prec0*recall0/(prec0+recall0) else 0

  c(accuracy = (tp + tn) / length(y_true),
    recall1 = recall1, precision1 = prec1,
    recall0 = recall0, precision0 = prec0,
    macro_f1 = (f1_1 + f1_0) / 2)
}

print_confusion <- function(y_true, y_pred, title = "MA TRAN NHAM LAN") {
  tp <- sum(y_true == 1 & y_pred == 1); tn <- sum(y_true == 0 & y_pred == 0)
  fp <- sum(y_true == 0 & y_pred == 1); fn <- sum(y_true == 1 & y_pred == 0)

  cat("\n", title, "\n", sep = "")
  print(matrix(c(tn, fp, fn, tp), nrow = 2, byrow = TRUE,
    dimnames = list(`Thực tế` = c("Không bệnh", "Có bệnh"),
    `Dự đoán` = c("Không bệnh", "Có bệnh"))))
  cat(sprintf(" TN = %3d không bệnh, dự đoán đúng\n", tn))
  cat(sprintf(" FP = %3d không bệnh, dự đoán nhầm là có bệnh\n", fp))
  cat(sprintf(" FN = %3d có bệnh, bị bỏ sót\n", fn))
  cat(sprintf(" TP = %3d có bệnh, phát hiện được\n", tp))
  invisible(compute_metrics(y_true, y_pred))
}
```

Macro F1, tức trung bình cộng chỉ số F1 của hai lớp, được chọn làm chỉ số tổng hợp vì nó gán trọng số bằng nhau cho hai lớp bất kể chênh lệch cỡ, phù hợp với đặc thù của bài toán mất cân bằng.

6 Bootstrap và Kiểm định chéo phân tầng (Stratified K-fold Cross-Validation)

Phần này xem xét hiệu năng của mô hình nền trước khi tinh chỉnh siêu tham số. Các giá trị được giữ cố định ở $\lambda = 1$, $k_{SMOTE} = 5$ và ngưỡng phân loại 0,5. Mục tiêu chưa phải là chọn cấu hình tốt nhất, mà là kiểm tra xem các chỉ số đánh giá có ổn định hay không khi dữ liệu dùng để đánh giá thay đổi.

6.1 Ước lượng khoảng tin cậy bằng Bootstrap

Recall của lớp có bệnh được tính từ một tập kiểm tra hữu hạn, nên giá trị này luôn đi kèm một mức bất định nhất định. Nếu tập kiểm tra gồm một nhóm bệnh nhân khác, dù vẫn được rút ra từ cùng quần thể, recall có thể thay đổi. Vì vậy,

bên cạnh ước lượng điểm, báo cáo dùng bootstrap để ước lượng khoảng tin cậy cho chỉ số này.

Cách thực hiện được giữ nhất quán với mục tiêu trên. Mô hình được huấn luyện một lần trên tập huấn luyện của phép chia 80:20, sau đó dự đoán trên tập kiểm tra. Trong các vòng bootstrap, mô hình không được huấn luyện lại; chỉ các cặp gồm nhãn thực tế và nhãn dự đoán trên tập kiểm tra được lấy mẫu lại có hoàn lại. Vì mô hình chỉ được huấn luyện một lần, khoảng tin cậy bootstrap chỉ phản ánh mức độ thay đổi của recall khi thành phần của tập kiểm tra thay đổi. Nó không phản ánh sự thay đổi của recall nếu mô hình được huấn luyện lại trên một tập huấn luyện khác.

```
sp <- stratified_split(y_all, test_frac = 0.2, seed = 42)
X_train <- X_all[sp$train, , drop = FALSE]; y_train <- y_all[sp$train]
X_test  <- X_all[sp$test, , drop = FALSE]; y_test  <- y_all[sp$test]

cat("Train:", nrow(X_train), "mẫu | Test:", nrow(X_test), "mẫu\n")

## Train: 467 mẫu | Test: 116 mẫu

# Huấn Luyện MỘT lần, dự đoán MỘT lần
fit_80 <- fit_pipeline(X_train, y_train, lambda = 1, smote_k = 5)
proba_test <- pipeline_proba(fit_80, X_test)
pred_test <- as.integer(proba_test >= 0.5)

recall_point <- compute_metrics(y_test, pred_test)["recall1"]

CONF <- 0.95
B <- 10000
n_test <- length(y_test)

set.seed(42)
recalls <- numeric(B)
n_valid <- 0
for (b in seq_len(B)) {
  # Rút có hoàn lại, cùng cỡ mẫu
  idx <- sample.int(n_test, n_test, replace = TRUE)
  if (sum(y_test[idx] == 1) == 0) next # không có ca bệnh -> recall không xác định
  n_valid <- n_valid + 1
  recalls[n_valid] <- compute_metrics(y_test[idx], pred_test[idx])["recall1"]
}
recalls <- recalls[seq_len(n_valid)]

# Khoảng tin cậy percentile: cắt (1 - CONF) / 2 ở mỗi đuôi
tail_pct <- (1 - CONF) / 2
ci <- quantile(recalls, c(tail_pct, 1 - tail_pct))

cat(sprintf("Recall lớp có bệnh   = %.3f\n", recall_point))

## Recall lớp có bệnh   = 0.614

cat(sprintf("Khoảng tin cậy %.0f%%   = [%.3f, %.3f]\n",
  CONF * 100, ci[1], ci[2]))

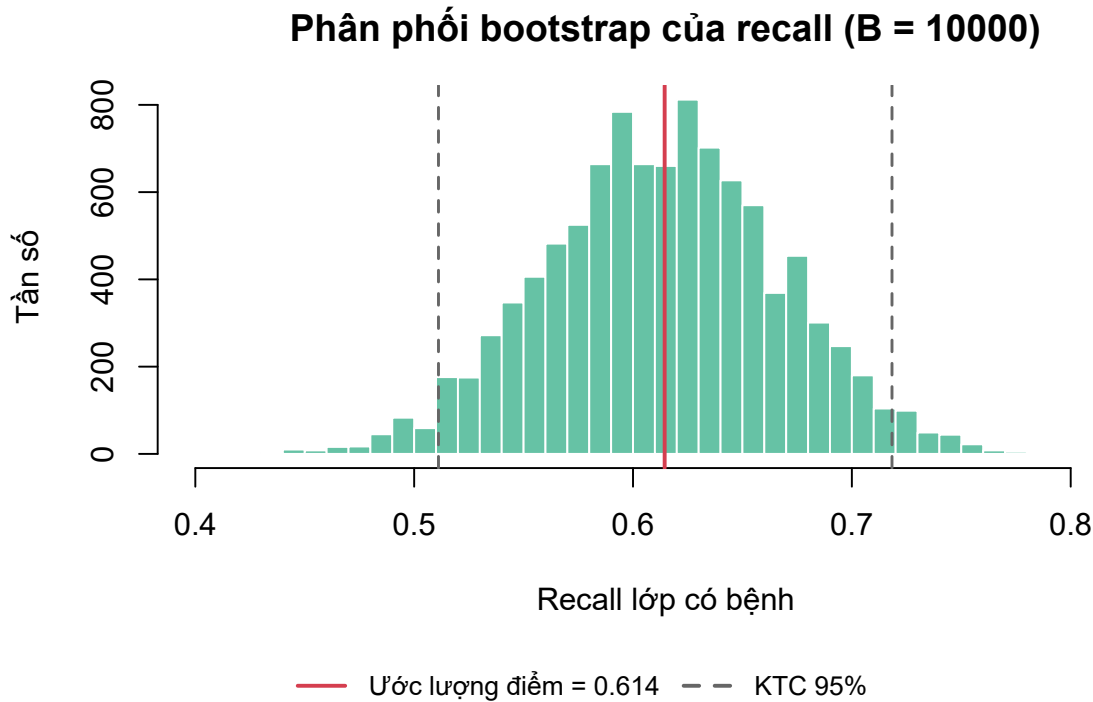
## Khoảng tin cậy 95%   = [0.511, 0.718]

cat(sprintf("Bề rộng khoảng       = %.3f\n", ci[2] - ci[1]))

## Bề rộng khoảng       = 0.207

cat(sprintf("Sai số chuẩn           = %.3f\n", sd(recalls)))

## Sai số chuẩn         = 0.054
```



Hình 5: Phân phối bootstrap của recall lớp có bệnh

Với ngưỡng phân loại mặc định 0,5, recall của lớp có bệnh đạt 0,614. Khoảng tin cậy bootstrap 95% là $[0,511; 0,718]$, có bề rộng 0,207. Khoảng tin cậy này cho thấy ước lượng recall vẫn còn mức độ biến thiên đáng kể do quá trình lấy mẫu từ tập kiểm tra. Điều này một phần phản ánh quy mô tập kiểm tra còn hạn chế (116 quan sát, trong đó có 83 ca bệnh), nên việc báo cáo khoảng tin cậy giúp mô tả đầy đủ hơn độ tin cậy của kết quả so với chỉ sử dụng một ước lượng điểm.

6.2 Kiểm định chéo phân tầng

Kết quả bootstrap ở phần trước được xây dựng từ một lần chia dữ liệu 80:20. Để đánh giá xem kết quả này có ổn định trước các cách chia dữ liệu khác nhau hay không, báo cáo tiếp tục sử dụng kiểm định chéo phân tầng 5-fold. Ở mỗi lượt, một fold được giữ lại làm tập kiểm tra, còn bốn fold còn lại được dùng để huấn luyện. Nhờ đó, mỗi quan sát đều được sử dụng đúng một lần để đánh giá và không xuất hiện trong tập huấn luyện của lần đánh giá đó.

Giá trị $K = 5$ được lựa chọn để mỗi fold kiểm tra chiếm khoảng 20% dữ liệu, tương ứng với phép chia 80:20 ở phần bootstrap. Nhờ vậy, quy mô của tập huấn luyện và tập kiểm tra giữa hai phương pháp gần tương đương, giúp việc so sánh kết quả trở nên hợp lý hơn.

```

K <- 5
folds <- stratified_folds(y_all, k = K, seed = 42)

cv_results <- data.frame()
for (i in seq_len(K)) {
  te <- which(folds == i)
  tr <- which(folds != i)

  fit_i <- fit_pipeline(X_all[tr, , drop = FALSE], y_all[tr],
                       lambda = 1, smote_k = 5)
  pred_i <- as.integer(pipeline_proba(fit_i,
                                     X_all[te, , drop = FALSE]) >= 0.5)

  m <- compute_metrics(y_all[te], pred_i)
  cv_results <- rbind(cv_results, data.frame(
    Fold = i, n_train = length(tr), n_test = length(te),
    Accuracy = m["accuracy"], Recall = m["recall1"],
    Macro_F1 = m["macro_f1"]
  ))
}
rownames(cv_results) <- NULL
cv_results[-1] <- round(cv_results[-1], 4)

```

Bảng 4: Kết quả kiểm định chéo 5 fold cho mô hình nền

Fold	n train	n test	Accuracy	Recall	Macro F1
1	465	118	0.6186	0.5238	0.6124
2	466	117	0.6923	0.6386	0.6776
3	467	116	0.6466	0.6145	0.6263
4	467	116	0.6466	0.6265	0.6230
5	467	116	0.6897	0.6265	0.6758

```

for (m in c("Accuracy", "Recall", "Macro_F1")) {
  cat(sprintf(" %-9s = %.3f +/- %.3f\n",
             m, mean(cv_results[[m]]), sd(cv_results[[m]])))
}

## Accuracy = 0.659 +/- 0.032
## Recall   = 0.606 +/- 0.047
## Macro_F1 = 0.643 +/- 0.031

```

Kết quả được báo cáo dưới dạng giá trị trung bình và độ lệch chuẩn giữa các fold. Trong ba chỉ số được xem xét, recall có độ biến động lớn nhất với giá trị $0,606 \pm 0,047$, dao động từ 0,524 đến 0,639 giữa các fold. Trong khi đó, macro F1 ổn định hơn, đạt $0,643 \pm 0,031$.

Recall trung bình của kiểm định chéo (0,606) cũng rất gần với recall thu được từ phép chia 80:20 (0,614). Đồng thời, cả hai giá trị đều nằm trong khoảng tin cậy bootstrap 95% [0,511; 0,718]. Mặc dù bootstrap và kiểm định chéo đánh giá hai khía cạnh khác nhau của hiệu năng mô hình, kết quả của chúng nhìn chung phù hợp với nhau và cùng cho thấy mô hình nền có mức hiệu năng tương đối ổn định, không bị chi phối bởi một cách chia dữ liệu cụ thể.

7 Tinh chỉnh siêu tham số

Từ bước này trở đi, tập kiểm tra được giữ riêng và chỉ dùng cho đánh giá cuối cùng. Việc tinh chỉnh siêu tham số được thực hiện hoàn toàn trên tập huấn luyện, thông qua kiểm định chéo phân tầng. Cách làm này giúp tránh tình trạng lựa chọn mô hình dựa trên thông tin của tập kiểm tra.

Hai siêu tham số cần lựa chọn là cường độ điều chuẩn λ của Ridge và số láng giềng k trong bước SMOTE. Lưới λ gồm ba giá trị $\{0,1; 1; 10\}$ đại diện cho mức điều chuẩn yếu, vừa và mạnh quanh vùng trung tâm. Kết hợp với ba giá trị $k_{SMOTE} \in \{3, 5, 7\}$, lưới tìm kiếm gồm 9 cấu hình, mỗi cấu hình được đánh giá bằng kiểm định chéo 5 fold trên tập huấn luyện, tương ứng với 45 lần khớp mô hình. Tiêu chí so sánh là macro F1 thay vì recall của lớp có bệnh. Nếu chỉ tối đa hóa recall, mô hình có thể đạt điểm rất cao bằng cách dự đoán gần như mọi quan sát là có bệnh; khi đó recall tăng, nhưng mô hình mất khả năng phân biệt giữa hai lớp. Macro F1 phù hợp hơn vì cân bằng đóng góp của cả lớp có bệnh và lớp không bệnh.

```
param_grid <- expand_grid(
  lambda = c(0.1, 1, 10),          # cường độ điều chuẩn Ridge
  smote_k = c(3, 5, 7)             # số láng giềng SMOTE
)

folds_inner <- stratified_folds(y_train, k = 5, seed = 42)

grid_mean <- grid_sd <- numeric(nrow(param_grid))
for (g in seq_len(nrow(param_grid))) {
  scores <- numeric(5)
  for (i in seq_len(5)) {
    te <- which(folds_inner == i); tr <- which(folds_inner != i)
    fit_g <- fit_pipeline(X_train[tr, , drop = FALSE], y_train[tr,
      lambda = param_grid$lambda[g],
      smote_k = param_grid$smote_k[g])
    pred_g <- as.integer(pipeline_proba(
      fit_g, X_train[te, , drop = FALSE]) >= 0.5)
    scores[i] <- compute_metrics(y_train[te], pred_g)["macro_f1"]
  }
  grid_mean[g] <- mean(scores)
  grid_sd[g] <- sd(scores)          # dao động giữa 5 fold, đo mức ổn định
}

param_grid$macro_f1 <- grid_mean
param_grid$sd_fold <- grid_sd

top5 <- head(param_grid[order(-param_grid$macro_f1), ], 5)
top5$macro_f1 <- round(top5$macro_f1, 4)
top5$sd_fold <- round(top5$sd_fold, 4)
```

Bảng 5: Năm cấu hình siêu tham số có macro F1 cao nhất (kiểm định chéo 5 fold)

Lambda	SMOTE k	Macro F1	SD giữa fold
1.0	7	0.6407	0.0314
10.0	7	0.6399	0.0294
0.1	3	0.6397	0.0243
0.1	7	0.6388	0.0301
10.0	5	0.6387	0.0285

Trong các cấu hình được khảo sát, cấu hình $\lambda = 1$ và $k_{\text{SMOTE}} = 7$ đạt macro F1 trung bình cao nhất, bằng 0,6407. Tuy nhiên, chênh lệch macro F1 giữa năm cấu hình dẫn đầu chỉ khoảng 0,002, trong khi độ lệch chuẩn giữa các fold đều xấp xỉ 0,03. Điều này cho thấy sự khác biệt về hiệu năng giữa các cấu hình là rất nhỏ và khó có thể xem là đáng kể.

```
i_top <- which.max(param_grid$macro_f1)
se_top <- param_grid$sd_fold[i_top] / sqrt(5) # sai số chuẩn của trung bình

# Nhóm cấu hình tương đương: cách điểm cao nhất không quá một sai số chuẩn
tied <- which(param_grid$macro_f1 >= param_grid$macro_f1[i_top] - se_top)

# Trong nhóm đó, ưu tiên lambda nhỏ nhất
cand <- tied[param_grid$lambda[tied] == min(param_grid$lambda[tied])]
best_idx <- cand[which.max(param_grid$macro_f1[cand])]
best <- param_grid[best_idx, ]

cat(sprintf("\nSai số chuẩn của trung bình 5 fold: %.4f\n", se_top))

##
## Sai số chuẩn của trung bình 5 fold: 0.0141

cat(sprintf("Số cấu hình tương đương: %d/%d\n",
            length(tied), nrow(param_grid)))

## Số cấu hình tương đương: 9/9

cat(sprintf("Cấu hình được chọn: lambda = %g, smote_k = %g (macro F1 = %.4f)\n",
            best$lambda, best$smote_k, best$macro_f1))

## Cấu hình được chọn: lambda = 0.1, smote_k = 3 (macro F1 = 0.6397)

cat(sprintf("Đã thử %d cấu hình x 5 fold = %d lần khớp mô hình\n",
            nrow(param_grid), nrow(param_grid) * 5))

## Đã thử 9 cấu hình x 5 fold = 45 lần khớp mô hình
```

8 Lựa chọn ngưỡng phân loại

Ngưỡng mặc định 0,5 phù hợp với các bài toán phân loại tổng quát, trong đó hai lớp được xem có tầm quan trọng như nhau. Tuy nhiên, trong chẩn đoán bệnh, bỏ

sốt một ca bệnh thường nghiêm trọng hơn nhiều so với việc báo động nhầm một người khỏe mạnh. Báo động nhầm chỉ dẫn đến các xét nghiệm bổ sung, trong khi bỏ sót có thể khiến bệnh không được phát hiện và điều trị kịp thời. Vì lẽ đó, ngưỡng phân loại thường được hạ xuống để tăng khả năng phát hiện các ca bệnh. Tuy vậy, nếu ngưỡng quá thấp, số người khỏe bị xếp nhầm vào nhóm có bệnh sẽ tăng nhanh, làm giảm độ tin cậy và giá trị ứng dụng của mô hình. Tóm lại, ngưỡng được lựa chọn là ngưỡng vừa giúp cải thiện đáng kể recall của lớp có bệnh, vừa giữ số báo động nhầm ở mức chấp nhận được, thay vì tối đa hóa recall bằng mọi giá.

Ngưỡng phân loại phải được lựa chọn hoàn toàn từ tập huấn luyện. Nếu dò ngưỡng trên tập kiểm tra thì thông tin từ tập này đã được sử dụng trong quá trình xây dựng mô hình, dẫn đến rò rỉ dữ liệu và làm kết quả đánh giá trở nên lạc quan hơn thực tế. Vì vậy, báo cáo sử dụng các dự đoán *out-of-fold* để lựa chọn ngưỡng. Cụ thể, mỗi quan sát trong tập huấn luyện được dự đoán bởi một mô hình chưa từng sử dụng chính quan sát đó để huấn luyện, nhờ vậy việc lựa chọn ngưỡng vẫn bảo đảm tính khách quan.

```
oof_proba <- numeric(length(y_train))
for (i in seq_len(5)) {
  te <- which(folds_inner == i); tr <- which(folds_inner != i)
  fit_o <- fit_pipeline(X_train[tr, , drop = FALSE], y_train[tr],
    lambda = best$lambda, smote_k = best$smote_k)
  oof_proba[te] <- pipeline_proba(fit_o, X_train[te, , drop = FALSE])
}

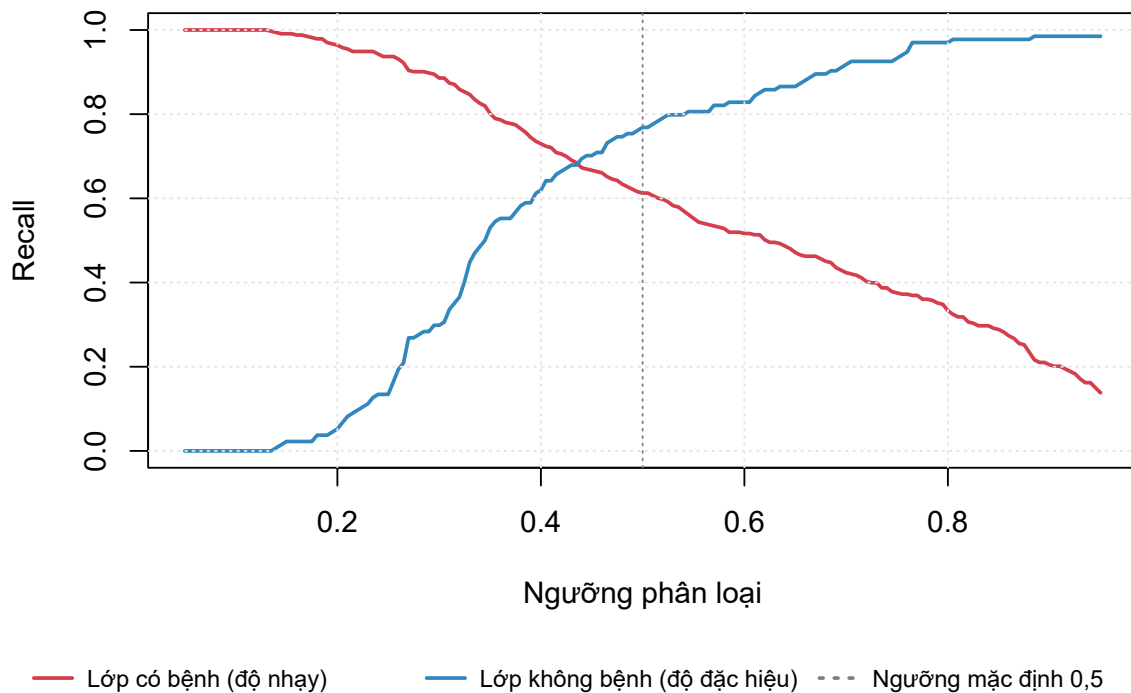
grid_thr <- seq(0.05, 0.95, by = 0.005)
rec1 <- rec0 <- n_fn <- n_fp <- numeric(length(grid_thr))
for (i in seq_along(grid_thr)) {
  pred_t <- as.integer(oof_proba >= grid_thr[i])
  m <- compute_metrics(y_train, pred_t)
  rec1[i] <- m["recall1"]; rec0[i] <- m["recall0"]
  n_fn[i] <- sum(y_train == 1 & pred_t == 0) # ca bệnh bị bỏ sót
  n_fp[i] <- sum(y_train == 0 & pred_t == 1) # người khỏe bị báo nhầm
}
```

```
# Chừa lề dưới cho chú giải: đặt ở "right" thì nó nằm đè lên chỗ hai đường
# recall cắt nhau, tức đúng vùng cần nhìn rõ nhất.
op <- par(mar = c(7, 4, 4, 2) + 0.1)
plot(grid_thr, rec1, type = "l", col = "#d53e4f", lwd = 2, ylim = c(0, 1),
  xlab = "Ngưỡng phân loại", ylab = "Recall",
  main = "Recall hai lớp theo ngưỡng (out-of-fold trên tập train)")
lines(grid_thr, rec0, col = "#3288bd", lwd = 2)
abline(v = 0.5, col = "gray50", lty = 3)
grid(col = "gray90")

# Bỏ tiền tố "Recall" trong nhãn vì trục tung đã ghi, nhờ vậy cả ba mục xếp
# vừa một hàng ngang.
legend(grconvertX(0.5, "ndc", "user"), grconvertY(0.05, "ndc", "user"),
  xjust = 0.5, yjust = 0.5, bty = "n", cex = 0.8, lwd = 2,
  legend = c("Lớp có bệnh (độ nhạy)",
    "Lớp không bệnh (độ đặc hiệu)",
    "Ngưỡng mặc định 0,5"),
  col = c("#d53e4f", "#3288bd", "gray50"), lty = c(1, 1, 3),
  horiz = TRUE, xpd = NA, seg.len = 1.8, x.intersp = 0.7)
```

Hình 6 minh họa mối quan hệ đánh đổi giữa recall của hai lớp khi thay đổi ngưỡng phân loại. Khi ngưỡng giảm, recall của lớp có bệnh tăng lên, nhưng

Recall hai lớp theo ngưỡng (out-of-fold trên tập train)



Hình 6: Đánh đổi giữa độ nhạy và độ đặc hiệu khi thay đổi ngưỡng phân loại

recall của lớp không bệnh giảm xuống. Vì vậy, việc lựa chọn ngưỡng thực chất là cân bằng giữa hai loại sai sót.

Bảng dưới đây liệt kê đủ sáu chỉ số trên các dự đoán out-of-fold của tập huấn luyện, tại các ngưỡng giảm dần từ ngưỡng mặc định 0,50 xuống 0,20 với bước nhảy 0,05.

```
coarse_thr <- seq(0.50, 0.20, by = -0.05)
coarse_tbl <- data.frame(nguonng = coarse_thr)
for (i in seq_along(coarse_thr)) {
  pr <- as.integer(oof_proba >= coarse_thr[i])
  m <- compute_metrics(y_train, pr)
  coarse_tbl$FN[i] <- sum(y_train == 1 & pr == 0)
  coarse_tbl$FP[i] <- sum(y_train == 0 & pr == 1)
  coarse_tbl$accuracy[i] <- round(unname(m["accuracy"]), 3)
  coarse_tbl$recall1[i] <- round(unname(m["recall1"]), 3)
  coarse_tbl$precision1[i] <- round(unname(m["precision1"]), 3)
  coarse_tbl$recall0[i] <- round(unname(m["recall0"]), 3)
  coarse_tbl$precision0[i] <- round(unname(m["precision0"]), 3)
  coarse_tbl$macro_f1[i] <- round(unname(m["macro_f1"]), 3)
}
```

Bảng 6: Các chỉ số trên out-of-fold theo ngưỡng, bước nhảy 0,05 từ 0,50 xuống 0,20

Ngưỡng	FN	FP	Accuracy	Recall 1	Precision 1	Recall 0	Precision 0	Macro F1
0.50	129	31	0.657	0.613	0.868	0.769	0.444	0.641
0.45	111	40	0.677	0.667	0.847	0.701	0.459	0.650
0.40	90	51	0.698	0.730	0.827	0.619	0.480	0.658
0.35	66	63	0.724	0.802	0.809	0.530	0.518	0.665
0.30	38	94	0.717	0.886	0.758	0.299	0.513	0.597
0.25	21	116	0.707	0.937	0.729	0.134	0.462	0.514
0.20	12	127	0.702	0.964	0.717	0.052	0.368	0.457

Bảng 6 cho thấy khi giảm ngưỡng phân loại, mô hình phát hiện được nhiều ca bệnh hơn: số ca bệnh bị bỏ sót (FN) giảm từ 129 xuống còn 12, đồng thời recall của lớp có bệnh tăng từ 0,613 lên 0,964. Tuy nhiên, đổi lại số người khỏe bị báo động nhầm (FP) tăng nhanh từ 31 lên 127, khiến khả năng nhận diện lớp không bệnh suy giảm đáng kể.

Ngưỡng 0,35 được lựa chọn vì tạo ra sự đánh đổi hợp lý giữa hai loại sai sót. So với ngưỡng mặc định, ngưỡng này giúp giảm đáng kể số ca bệnh bị bỏ sót, trong khi số người khỏe bị báo động nhầm chưa tăng đến mức làm mất giá trị ứng dụng của mô hình.

9 Đánh giá trên tập kiểm tra

Đây là lần đầu tiên tập kiểm tra được sử dụng kể từ đầu quy trình. Mô hình được huấn luyện lại trên toàn bộ tập huấn luyện với bộ siêu tham số đã chọn, sau đó áp dụng ngưỡng phân loại 0,350.

```
fit_best <- fit_pipeline(X_train, y_train,
                        lambda = best$lambda, smote_k = best$smote_k)
proba_final <- pipeline_proba(fit_best, X_test)
pred_final <- as.integer(proba_final >= threshold)
pred_05 <- as.integer(proba_final >= 0.5)

m_final <- print_confusion(y_test, pred_final,
                           sprintf("MA TRAN NHAM LAN (nguong %.3f)",
                                   threshold))

##
## MA TRAN NHAM LAN (nguong 0.350)
##          Dự đoán
## Thực tế  Không bệnh  Có bệnh
## Không bệnh      18      15
## Có bệnh         21      62
## TN = 18 không bệnh, dự đoán đúng
## FP = 15 không bệnh, dự đoán nhầm là có bệnh
## FN = 21 có bệnh, bị bỏ sót
## TP = 62 có bệnh, phát hiện được
```

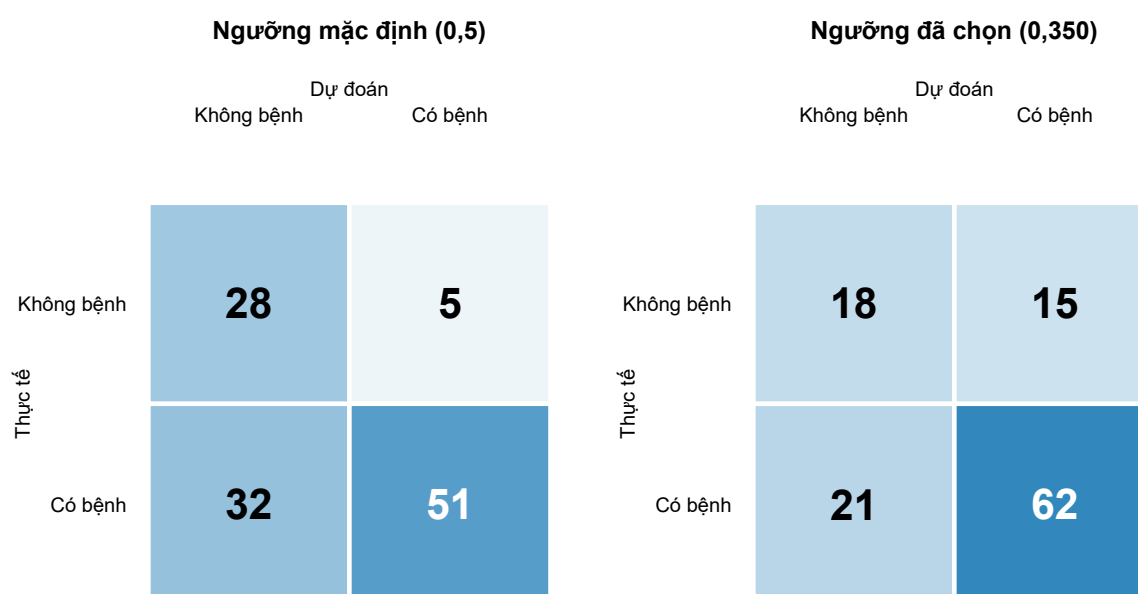
Bảng 7: Các chỉ số trên tập kiểm tra với ngưỡng đã chọn

Accuracy	Recall lớp 1	Precision lớp 1	Recall lớp 0	Precision lớp 0	Macro F1
0.69	0.747	0.805	0.545	0.462	0.637

```
cmp <- rbind(
  `0.500 (mặc định)` = compute_metrics(y_test, pred_05),
  `đã chỉnh`         = compute_metrics(y_test, pred_final)
)
cmp_tbl <- data.frame(Nguong = rownames(cmp), round(as.data.frame(cmp), 3),
  row.names = NULL)
```

Bảng 8: So sánh các chỉ số giữa ngưỡng mặc định và ngưỡng đã chọn

Ngưỡng	Accuracy	Recall lớp 1	Precision lớp 1	Recall lớp 0	Precision lớp 0	Macro F1
0.500 (mặc định)	0.681	0.614	0.911	0.848	0.467	0.668
đã chỉnh	0.690	0.747	0.805	0.545	0.462	0.637



Hình 7: Ma trận nhầm lẫn trên tập kiểm tra: ngưỡng mặc định so với ngưỡng đã chọn

Hình 7 trực quan hoá hai ma trận nhầm lẫn cùng lúc, với sắc độ đậm nhạt của ô phản ánh số lượng quan sát rơi vào ô đó. So với ngưỡng mặc định 0,500, ngưỡng 0,350 làm giảm số ca bệnh bị bỏ sót từ 32 xuống 21, tức phát hiện thêm 11 bệnh nhân mắc bệnh. Đổi lại, số người khỏe bị báo động nhầm tăng từ 5 lên

15. Recall của lớp có bệnh tăng từ 0,614 lên 0,747, trong khi độ chính xác tổng thể gần như không đổi, từ 0,681 lên 0,690.

Hai loại sai sót không còn nghiêng hẳn về phía bỏ sót ca bệnh như ở ngưỡng mặc định (32 so với 5), mà khoảng cách giữa chúng đã thu hẹp đáng kể (21 so với 15). Recall và precision của lớp có bệnh lần lượt là 0,747 và 0,805; recall và precision của lớp không bệnh lần lượt là 0,545 và 0,462. Cái giá phải trả là độ đặc hiệu giảm khá nhiều, từ 0,848 xuống 0,545, còn macro F1 giảm nhẹ từ 0,668 xuống 0,637.

Cả hai chỉ số đều khớp khá tốt giữa hai tập. Recall của lớp có bệnh đạt 0,802 trên các dự đoán *out-of-fold* của tập huấn luyện so với 0,747 trên tập kiểm tra, còn độ đặc hiệu là 0,530 trên tập huấn luyện so với 0,545 trên tập kiểm tra. Sai lệch vài điểm phần trăm giữa hai tập là mức dao động bình thường của một tập kiểm tra chỉ có 116 quan sát, và cho thấy ngưỡng dò được trên tập huấn luyện tái lập khá tốt trên dữ liệu chưa từng được dùng để chọn nó.

Tuy nhiên, các chỉ số trên đều là những ước lượng điểm được tính từ tập kiểm tra gồm 116 quan sát nên chưa phản ánh mức độ bất định của kết quả. Vì vậy, bootstrap 10.000 vòng được áp dụng theo đúng quy trình đã trình bày ở Mục 6.1 để ước lượng khoảng tin cậy cho toàn bộ sáu chỉ số đánh giá.

```
CONF2 <- 0.95
B2 <- 10000
boot_mat <- matrix(NA, nrow = B2, ncol = length(m_final),
  dimnames = list(NULL, names(m_final)))

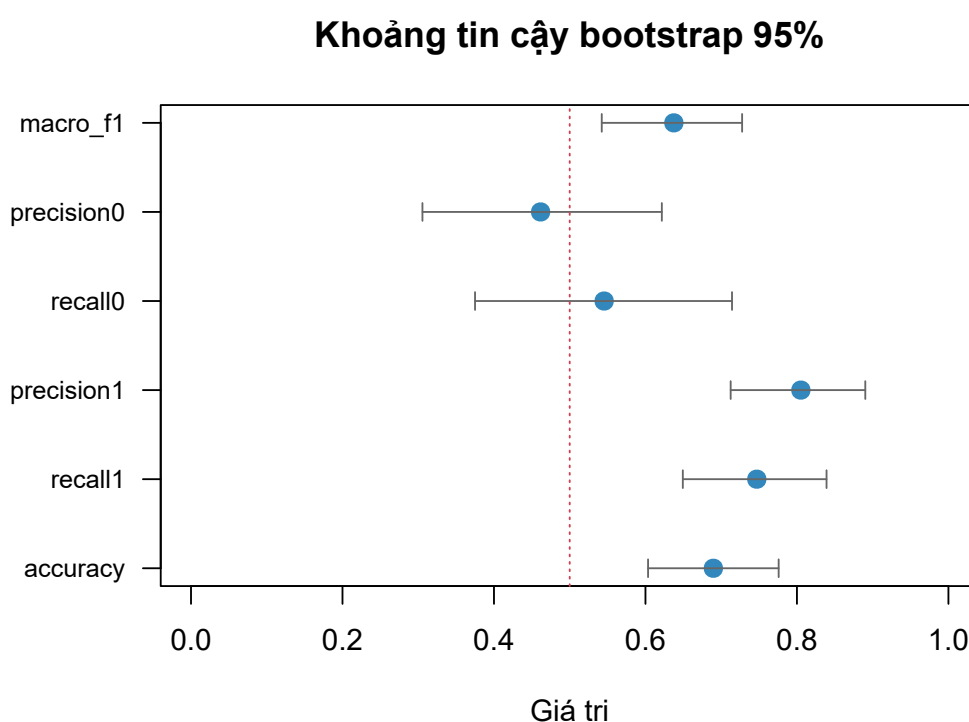
for (b in seq_len(B2)) {
  idx <- sample.int(n_test, n_test, replace = TRUE)
  if (length(unique(y_test[idx])) < 2) next # mẫu suy biến, bỏ qua
  boot_mat[b, ] <- compute_metrics(y_test[idx], pred_final[idx])
}
boot_mat <- boot_mat[complete.cases(boot_mat), , drop = FALSE]

tail2 <- (1 - CONF2) / 2
ci_tbl <- data.frame(
  diem_uoc_luong = m_final,
  ci_duoi = apply(boot_mat, 2, quantile, probs = tail2),
  ci_tren = apply(boot_mat, 2, quantile, probs = 1 - tail2),
  sai_so_chuan = apply(boot_mat, 2, sd)
)
```

Bảng 9: Khoảng tin cậy bootstrap 95% cho sáu chỉ số trên tập kiểm tra

Chỉ số	Ước lượng điểm	Cận dưới	Cận trên	Sai số chuẩn
accuracy	0.6897	0.6034	0.7759	0.0433
recall1	0.7470	0.6493	0.8391	0.0482
precision1	0.8052	0.7125	0.8902	0.0456
recall0	0.5455	0.3750	0.7143	0.0878
precision0	0.4615	0.3056	0.6216	0.0806
macro_f1	0.6375	0.5423	0.7276	0.0478

Recall của lớp có bệnh đạt 0,747, với khoảng tin cậy bootstrap 95% [0,649; 0,839]. Hình 8 cho thấy độ rộng của các khoảng tin cậy khác nhau đáng kể giữa các chỉ số. Accuracy có khoảng tin cậy hẹp nhất, với sai số chuẩn 0,043, vì được tính trên toàn bộ 116 quan sát của tập kiểm tra. Ngược lại, độ đặc hiệu (recall của lớp không bệnh) có khoảng tin cậy rộng nhất, [0,375; 0,714], với sai số chuẩn 0,088, do chỉ dựa trên 33 người khoẻ trong tập kiểm tra; precision của lớp không bệnh cũng có khoảng tin cậy rộng tương tự, [0,306; 0,622], sai số chuẩn 0,081, vì chỉ dựa trên 39 dự đoán âm tính.



Hình 8: Khoảng tin cậy bootstrap 95% cho sáu chỉ số trên tập kiểm tra

10 Kết luận

Báo cáo đã xây dựng một quy trình phân loại hoàn chỉnh trên bộ dữ liệu ILPD và, quan trọng hơn, minh họa sự phân công vai trò rõ ràng giữa hai kỹ thuật resampling.

Kiểm định chéo phân tầng 5 phần được dùng ở ba vị trí: đánh giá mô hình cơ sở (macro F1 $0,643 \pm 0,031$), dò 9 cấu hình siêu tham số, và sinh dự đoán out-of-fold để chọn ngưỡng phân loại. Độ lệch chuẩn giữa các fold cho thấy cả 9 cấu hình nằm trong phạm vi một sai số chuẩn của nhau — khác biệt giữa chúng không đáng kể, điều một lần chia train/test đơn lẻ không thể phát hiện.

Bootstrap cho thấy recall 0,747 trên tập kiểm tra đi kèm khoảng tin cậy 95% khá rộng, $[0,649; 0,839]$ (gần 19 điểm phần trăm), nghĩa là với cỡ mẫu hiện có không thể phân biệt mô hình này với một mô hình có recall thực từ 0,65 đến 0,84.

Mô hình cuối cùng là hồi quy logistic Ridge ($\lambda = 0,1$) kết hợp SMOTE ($k = 3$) trên 7 biến còn lại sau khi loại Direct_Bilirubin (đa cộng tuyến), Gender và Total_Protiens (không đạt ngưỡng ý nghĩa sau FDR); nhóm biến quan trọng nhất là men gan và bilirubin. Ngưỡng phân loại 0,350 được chọn vì cân bằng hợp lý hai loại sai sót trên out-of-fold. Trên tập kiểm tra, kết quả là 21 ca bệnh bị bỏ sót và 15 người khoẻ bị báo nhầm (so với 32 và 5 ở ngưỡng mặc định), đổi lại độ đặc hiệu giảm từ 0,848 xuống 0,545 và macro F1 giảm nhẹ xuống 0,637.

10.1 Hạn chế

1. Hiệu năng tuyệt đối còn thấp: bỏ sót 21/83 bệnh nhân, báo nhầm 15/33 người khoẻ; precision lớp không bệnh chỉ 0,462.
2. Khoảng tin cậy bootstrap chỉ phản ánh biến thiên do lấy mẫu tập kiểm tra, chưa tính biến thiên do thay đổi tập huấn luyện, nên độ bất định thực tế rộng hơn con số đã công bố.
3. Cỡ mẫu nhỏ (583 quan sát, 116 dùng để kiểm tra) khiến khoảng tin cậy rộng và nhiều so sánh không đạt mức phân biệt thống kê.
4. Dữ liệu chỉ thu thập tại một vùng địa lý duy nhất (đông bắc bang Andhra Pradesh, Ấn Độ), nên kết quả chưa chắc tổng quát hoá được.
5. Albumin và Albumin_and_Globulin_Ratio vẫn đa cộng tuyến về định nghĩa; Ridge làm giảm nhưng không loại bỏ hoàn toàn ảnh hưởng này, nên các hệ số riêng lẻ không nên được diễn giải như mức độ quan trọng của từng biến.

10.2 Hướng mở rộng

Ba hướng ưu tiên: huấn luyện lại mô hình trong từng vòng bootstrap để khoảng tin cậy bao quát cả biến thiên huấn luyện; lặp lại kiểm định chéo với nhiều cách chia để kiểm tra độ ổn định của kết luận về 9 cấu hình; và mở rộng sang các mô hình phi tuyến (random forest, gradient boosting) có khả năng nắm bắt tương tác giữa các chỉ số men gan.

Tài liệu tham khảo

1. Benjamini, Y., & Hochberg, Y. (1995). Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. *Journal of the Royal Statistical Society, Series B*, 57(1), 289–300.
2. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
3. Efron, B. (1979). Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics*, 7(1), 1–26.
4. Efron, B., & Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall.
5. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.
6. Hoerl, A. E., & Kennard, R. W. (1970). Ridge Regression: Biased Estimation for Nonorthogonal Problems. *Technometrics*, 12(1), 55–67.
7. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning*. Springer.
8. Kohavi, R. (1995). A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection. *Proceedings of IJCAI*, 1137–1143.
9. R Core Team (2026). *R: A Language and Environment for Statistical Computing* (version 4.6.1). R Foundation for Statistical Computing, Vienna, Austria.
10. Ramana, B., & Venkateswarlu, N. (2022). ILPD (Indian Liver Patient Dataset) [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5D02C>